

The proofgraph package

Pierre Senellart
pierre@senellart.com

2026/07/31 v1.1.0

Abstract

The `proofgraph` package automatically produces a graph of the dependencies between the results (theorems, lemmas, propositions) of a mathematical article. It infers an edge from one result to another whenever the proof of the former refers to the latter through an ordinary cross-reference, so that most dependencies need no manual annotation; commands are provided for those a visible reference does not capture. The graph is written as a `Graphviz .dot` file, which can be rendered externally or, optionally, embedded into the document automatically.

1 Introduction

When writing or reading a mathematical article, it is often useful to visualise how the results depend on one another: which lemma is used in the proof of which theorem, and so on. `proofgraph` builds such a dependency graph automatically. Each theorem-like environment becomes a node; each cross-reference (`\ref`, `\cref`...) appearing *inside a proof* becomes an edge from the proved result to the referred-to result. Usually no manual annotation is required, and commands are provided for the dependencies a visible reference does not capture.

The package is best explained by a small example of its use. This manual is itself a `proofgraph` document, so the results stated below are typeset with the package enabled, and the graph shown at the end of this section is the one they produce.

Load `proofgraph` in the preamble, before the `\newtheorem` commands that declare your theorem-like environments (so it sees them as they are created). Here, as in this manual, we pass three options: `autorun` renders the graph with `Graphviz` and embeds it automatically (it needs shell-escape); `cite` captures the `\cites` made inside proofs as dependencies on external work; and `citelabel=tag` labels those citation nodes by their printed tag:

```
\usepackage[autorun,cite,citelabel=tag]{proofgraph}
\newtheorem{theorem}{Theorem}
\newtheorem{lemma}{Lemma}
\newtheorem{corollary}{Corollary}
```

Then state and prove, as usual, a few results that build on one another:

```
\begin{lemma}\label{lem:even}
  The sum of two even integers is even.
\end{lemma}
```

```

\begin{lemma}\label{lem:odd}
  The sum of two odd integers is even.
\end{lemma}
\begin{theorem}\label{thm:parity}
  The sum of two integers of the same parity is even.
\end{theorem}
\begin{proof}
  By Lemmas~\ref{lem:even} and~\ref{lem:odd}; see~\cite{euclid}.
\end{proof}
\begin{corollary}\label{cor:double}
  For every integer  $n$ , the number  $2n$  is even.
\end{corollary}
\begin{proof}
  Apply Theorem~\ref{thm:parity} to  $n$  and  $n$ .
\end{proof}

```

We obtain the following results, exactly as without the package:

Lemma 1. *The sum of two even integers is even.*

Lemma 2. *The sum of two odd integers is even.*

Theorem 1. *The sum of two integers of the same parity is even.*

Proof. By Lemmas 1 and 2; see [1]. □

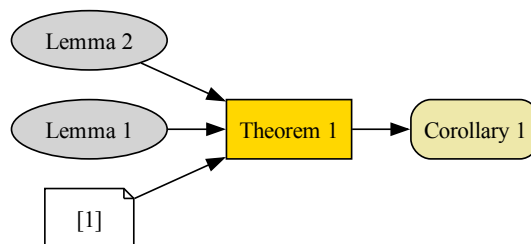
Corollary 1. *For every integer n , the number $2n$ is even.*

Proof. Apply Theorem 1 to n and n . □

In addition, `proofgraph` writes out the dependency graph. With the `autorun` option it is rendered with `Graphviz` and included wherever you call `\proofgraph`:

```
\proofgraph[width=0.7\textwidth]
```

which here produces:



Each cross-reference made inside a proof becomes an arrow *into* the result that proof establishes: the two lemmas point to Theorem 1, which in turn points to Corollary 1. The `\cite` in the proof of Theorem 1 is captured in the same way, thanks to the `cite` option: the external work appears as a distinct node (drawn note-shaped, and labelled here by its printed citation tag, “[1]”, because of `citelabel=tag`) with an arrow into the theorem it helps prove. Moreover, since

this manual loads `hyperref` and `TikZ`, the graph’s nodes are *clickable*: try clicking one to jump to the result it represents (see `\proofgraph` and the `hyperlinks` option).

A handful of commands tune such a graph: `\proofgraphstyle` sets the appearance of each kind of node (the colours here come from it); `\uses` and `\proofgraphedge` record a dependency that no visible `\ref` expresses; `\proofof` attaches a proof to its result; `\proofgraphuntrack` keeps an environment out of the graph; and `\proofgraphignore` and `\proofgraphexclude` drop an unwanted edge or node. The next section puts several of these to work on a real article; Section 3 then documents them, and the package options, in full.

2 A real-world example

The introductory example is deliberately tiny. To show what `proofgraph` produces on a genuine article, the graph in Figure 1 is the one obtained from *Connecting Knowledge Compilation Classes and Width Parameters* [2]. It has 62 numbered results, drawn from five environments (`result`, `theorem`, `proposition`, `corollary` and `lemma`), depends on 24 external works, and contains 114 edges in all.

The graph was produced from the paper essentially as published. Beyond loading the package, the only additions were the following.

- The `cite` option was switched on, and citation nodes labelled by their printed tag, by loading the package as `\usepackage[cite=true,citelabel=tag]{proofgraph}`. The `\cites` made inside proofs then appear as the note-shaped nodes (“[17]”, “[46, Theorem 4.10]”...) along the left of the graph; a work cited for two different results, or for two of its own theorems, gives one node per `\cite` note, so those 23 works appear as 39 nodes.
- One environment was kept out of the graph with a single `\proofgraphuntrack{observation}` (a lone preliminary observation that nothing else uses).
- The five result kinds were styled, with decreasing prominence, by five `\proofgraphstyle` declarations:

```
\proofgraphstyle{result}{shape=box,style="filled,bold",%
                        fillcolor=orange,peripheries=2}
\proofgraphstyle{theorem}{shape=box,style=filled,%
                        fillcolor=gold,penwidth=2}
\proofgraphstyle{corollary}{shape=box,style="rounded,filled",%
                        fillcolor=palegoldenrod}
\proofgraphstyle{proposition}{shape=box,style="rounded,filled",%
                        fillcolor=lightblue}
\proofgraphstyle{lemma}{shape=ellipse,style=filled,%
                        fillcolor=lightgray}
```

- Eleven `\proofgraphedge` commands, recording 27 edges in total, were added for the dependencies that no `\ref` inside a `proof` expresses: eight “obvious” corollaries stated without a `proof` environment (`\proofgraphedge{cor:upper_bound}{thm:upper_bound,...}`), and

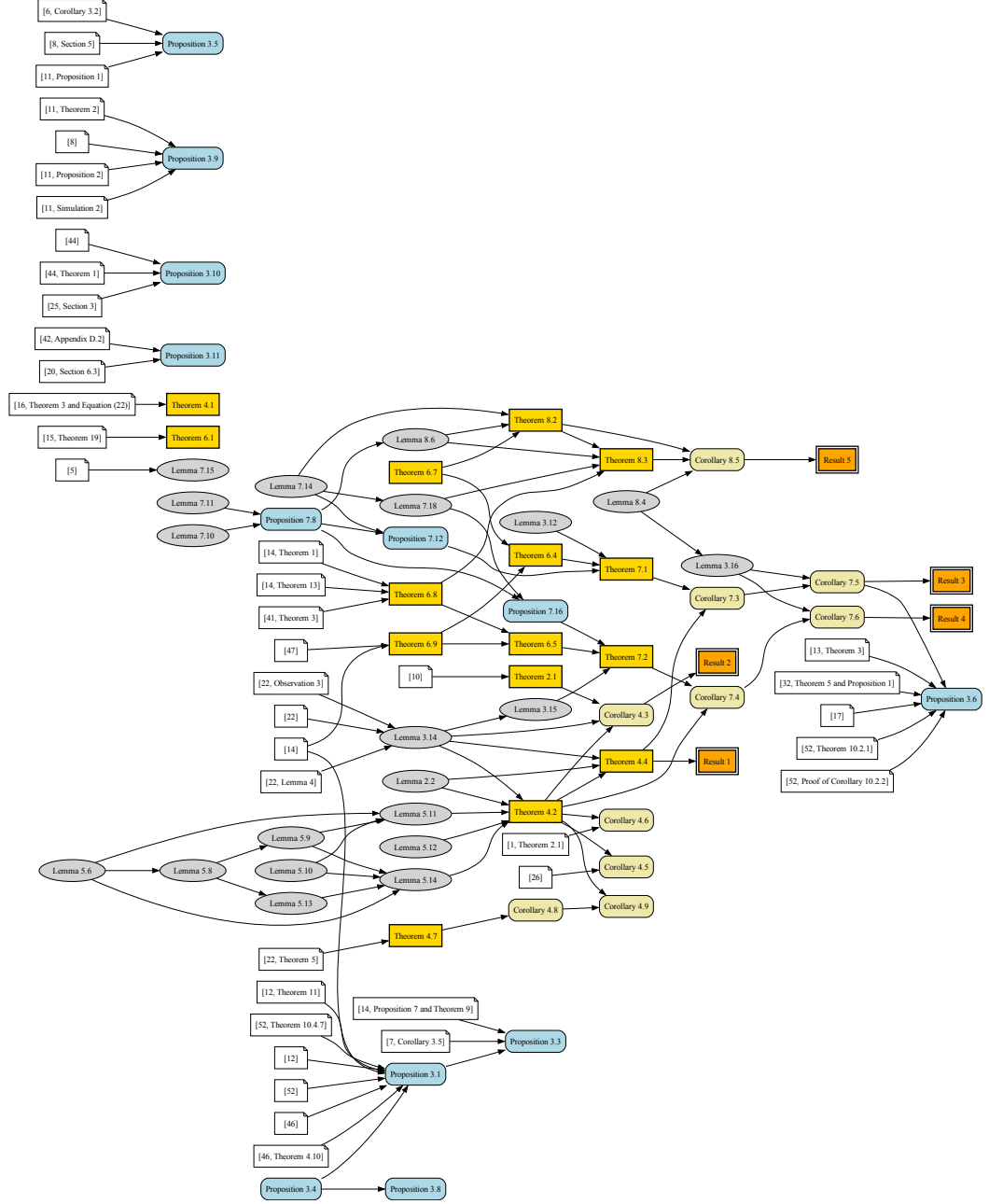


Figure 1: The graph **proofgraph** produces for [2]. The colours and shapes make the structure legible, with the orange double-bordered introduction **results** on the right, fed by the gold theorems, the blue and pale-gold propositions and corollaries, and the grey lemmas.

three theorems whose proof is developed across a whole section rather than inside a single `proof`.

- One `\proofgraphciteedge`, for a lemma stated without a proof, its proof being in the conference version of the article.

Nothing else was annotated: every other edge, and the numbering of all 62 nodes, was inferred automatically from the ordinary cross-references the proofs already contained. In particular, no `\uses`, `\proofof`, `\proofgraphignore` or `\proofgraphexclude` was needed. The five `result` environments that restate the headline results in the introduction (`\begin{result}[(Corollary~\ref{cor:obdd_omega})]...`) are linked to the body results they preview through the `\ref` in their optional title, captured automatically (see the note on reference-carrying titles in Section 3). Two compilation runs (plus `BBTEX`, needed once for the `tag` labels) suffice; the `.dot` file is then rendered with `Graphviz, dot -Tpdf main.dot -o main.pdf`, as with any other document.

3 Usage

3.1 Default behaviour

Load `proofgraph` in the preamble. It only needs to come *after* whatever provides your theorem environments, that is, after `\usepackage{amsthm}` if you load it yourself (some classes, such as `lipics` or `acmart`, provide theorems for you, in which case there is nothing to load first), and *before* the `\newtheorem` commands that declare your theorem-like environments, so that `proofgraph` sees them as they are created:

```
\usepackage{amsthm}      % or nothing, if your class provides theorems
\usepackage{proofgraph}
\newtheorem{theorem}{Theorem}
\newtheorem{lemma}{Lemma}
```

Then write your document as usual. After two compilation runs, a file `\jobname.dot` is produced. Render it with `Graphviz`:

```
dot -Tpdf myfile.dot -o mygraph.pdf
```

Every *numbered* result of a theorem-like environment you declare becomes a node, and every cross-reference (`\ref`, `\cref`...) inside a proof becomes an edge into the result that proof establishes, with no annotation. The commands captured are `\ref`, `\cref` and `\Cref` (`cleveref`), `\autoref` (`hyperref`), `\eqref` (`amsmath`), `\vref` (`varioref`) and `\zceref` (`zref-clever`, in every form: starred, with options, and with a comma-separated list of labels), each of them whenever it is defined. Purely page-related references (`\pageref`, `\cpageref`, `\zcpageref`) are not captured, as they point at a place rather than at a result; `\uses` records a dependency they would have expressed. Nodes are labelled by the result's formatted name and number ("Theorem 1"), not the environment name. Unnumbered (starred) results are skipped, as they are usually expository or repeated statements; `\proofgraphtrack` draws a chosen one anyway. A cross-reference in the *optional title* of a result is captured the same way: `\begin{theorem}[(generalises~\ref{thm:x})]` in a result labelled $\langle A \rangle$ adds

an edge from $\langle A \rangle$ to `thm:x`, handy for restatements (e.g., an introduction that previews results with `\begin{result}[(Corollary~\ref{cor:y})]`). The *body* of a result is read the same way: a statement that refers to an earlier result, “Under the hypotheses of Theorem 2...”, depends on it just as a proof would. Set `statements=false` to build the graph from proofs alone.

A result *stated inside a proof*, a claim raised and settled in the course of an argument, is drawn as a node of its own and given an edge to the result that proof establishes: the claim is a step of that proof, and the graph says so without any annotation. Its own proof, whether it follows the claim as a nested **proof** environment or as the rest of the enclosing one, hangs below the claim, so the argument appears in the graph with the shape it has on the page. An unnumbered claim, like any unnumbered result, is left out, and its dependencies with it; number it, or name it in `\proofgraphtrack`, to have it drawn.

3.2 Recording dependencies by hand

`\uses{<label>}` Inside a proof, records a dependency on the result labelled $\langle label \rangle$ even when no visible reference is made. A comma-separated list is accepted, as in `\uses{lem:a,lem:b}`.

`\proofgraphedge{<A>}{<labels>}` Adds edges recording that result $\langle A \rangle$ depends on the result(s) $\langle labels \rangle$ (a comma-separated list), like a `\uses` but stated outside any proof. Handy for an “obvious” corollary that relies on earlier results without a **proof** environment, as in `\proofgraphedge{cor:x}{thm:y,lem:z}`. It may appear anywhere; all labels are resolved when the graph is written.

`\usescite[<note>]{<keys>}` Inside a proof, the `\uses` of external work: records a dependency on the bibliography entries $\langle keys \rangle$ (a comma-separated list) and draws them as citation nodes, without citing them visibly. The optional $\langle note \rangle$ is the one `\cite` would have taken, as in `\usescite[Thm~3]{knuth}`; it labels the node and, as with a captured `\cite`, tells one result of a work from another.

`\proofgraphciteedge[<note>]{<A>}{<keys>}` The `\proofgraphedge` of citation nodes: records that result $\langle A \rangle$ depends on the cited work(s) $\langle keys \rangle$, stated outside any proof, as in `\proofgraphciteedge{thm:x}{knuth}`. Useful for a result stated without a proof, which has no place to hold a citation. Like `\proofgraphedge` it may appear anywhere, all labels being resolved when the graph is written.

Both commands create the citation node when the work has none yet, and both work whether or not the `\cite` option is on: that option only says whether a plain `\cite` is captured on its own. A work reached both ways, once by `\cite` capture and once by hand, still yields a single node.

`\proofof{<label>}` By default a proof is attached to the most recently started result. A detached proof whose optional title names its result is attached automatically: `\begin{proof}` with `[of Theorem~\ref{thm:x}]` attaches the proof to `thm:x` (for the `amsthm` proof environment, for **proof** defined as a theorem in the Springer classes, and for deferred proofs as with `apxproof`). When none of this applies, `\proofof` inside the proof pins it to the result

labelled $\langle label \rangle$. It pins the proof *as a whole*, wherever in it it stands, so the references made before it count as much as those made after; $\langle label \rangle$ may be the label of a result stated later in the document. This is what a proof written in an appendix, or anywhere else away from its result, needs and all that it needs: its dependencies are then captured as those of a proof placed right after its result would be, and recording them by hand with `\uses` or `\proofgraphedge` is not called for.

The pin covers the proofs *nested* in the pinned one as well, the steps of a structured proof (as `pf2` writes them) being proofs in their own right; a `\proofof` in a nested proof pins that proof alone, overriding for it the pin of the proof around it. A nested proof that comes after a *result* opened inside the enclosing proof – the proof of a claim stated in the course of an argument – attaches to that claim instead, as an ordinary proof would.

3.3 Choosing what appears in the graph

Result environments are detected automatically; these commands adjust which results, and which edges, are kept.

`\proofgraphtrack{\langle names \rangle}` Forces the listed environment types to be tracked, overriding the default exclusions, which are `definition`, `example`, `remark` and `note`. Use it in the preamble.

`\proofgraphuntrack{\langle names \rangle}` Excludes further environment types, as in `\proofgraphuntrack{observation}`. Use it in the preamble.

`\proofgraphproofenv{\langle names \rangle}` Treats the listed environments as proofs, besides `proof`, so that the references they make become dependencies of the result they establish. Use it in the preamble, as in `\proofgraphproofenv{pf}`. It serves a document whose proofs live in an environment of another name, for instance because a proof package had to be renamed to avoid the clash with `amsthm`'s `proof` (as with `pf2`), and equally a `\newtheorem{claimproof}` used for the proofs of claims. Being proofs, the environments named are also removed from the results drawn, even when they are numbered; a `\proofgraphtrack` on the same name overrides that, for an environment to be drawn as a result as well.

An environment taking an optional title is wrapped as `proof` is, so a detached `\begin{pf}[of Theorem~\ref{thm:x}]` attaches to `thm:x`; one declared with `\newtheorem` attaches to the result it follows, or to the one `\proofof` names.

`\proofgraphignore{\langle A \rangle}{\langle B \rangle}` Drops the dependency of the result $\langle A \rangle$ on the result $\langle B \rangle$, the edge the proof of $\langle A \rangle$ produced by referring to $\langle B \rangle$: an unwanted forward reference, say. It may appear anywhere, the preamble included.

The two labels are those of `\proofgraphedge`, in that order: the result first, what its proof uses second, the way the dependency is *recorded*. They are *not* the ends of the arrow *drawn*, which with the default `direction=usedby` runs the other way, from $\langle B \rangle$ to $\langle A \rangle$. So the rule undoing a `\proofgraphedge{cor:x}{thm:y}` is

`\proofgraphignore{cor:x}{thm:y}`, with the same two labels in the same order, and the rule dropping the arrow the picture draws from `lem:a` to `thm:m` is `\proofgraphignore{thm:m}{lem:a}`. The `direction` option does not enter into it: like every filter the rule works on the dependency recorded, `direction` settling only how it is drawn, so a graph turned round keeps its rules. A rule that drops nothing is reported at the end of the run, and says so when it is the two labels that have been swapped.

`\proofgraphignorecite[⟨note⟩]{⟨A⟩}{⟨keys⟩}` The same for a citation node, which `\proofgraphignore` cannot name, both of its arguments being labels: drops the dependency of the result `⟨A⟩` on the cited works `⟨keys⟩` (a comma-separated list of BibTeX keys). Its arguments are those of `\proofgraphciteedge`, which it undoes. The optional `⟨note⟩` picks, of a work cited with a note and so drawn as more than one node, the node of that note, so `\proofgraphignorecite[Thm~3]{thm:x}{knuth}` leaves the node of a plain `\cite{knuth}` alone. A citation node no edge reaches is not drawn, so dropping the last dependency on a work removes its node with it.

`\proofgraphexclude{⟨label⟩}` Removes the result labelled `⟨label⟩`, together with all its incident edges, from the graph.

`\proofgraphcitecommand{⟨names⟩}` Captures further citation commands, given as a comma-separated list of names without their backslash, as in `\proofgraphcitecommand{citeauthor,citeyear}`. Use it in the preamble. Any command taking the usual `⟨cs⟩* [⟨pre⟩] [⟨post⟩] {⟨keys⟩}` arguments works, including one you define yourself. Naming a command the document does not define is harmless.

It cannot help with the biblatex *multicite* commands (`\cites`, `\parencites`, `\textcites`), whose `(⟨pre⟩)(⟨post⟩)` syntax and repeated key groups `proofgraph` does not parse; cite such works individually, or record them with `\uses`.

3.4 Styling the graph

The attributes are passed verbatim to Graphviz; see its attribute reference, <https://graphviz.org/doc/info/attrs.html>, for the node attributes available (and <https://graphviz.org/doc/info/shapes.html> for the shapes).

`\proofgraphstyle{⟨env⟩}{⟨attrs⟩}` Sets Graphviz node attributes for all results of the environment `⟨env⟩`, e.g., `shape=ellipse` or `style=filled,fillcolor=gold`.

`\proofgraphstylecite{⟨attrs⟩}` Sets the Graphviz attributes of the external citation nodes added by the `cite` option (default `shape=note`), as in `\proofgraphstylecite{shape=box,style=dashed,color=gray}`.

3.5 Embedding the rendered graph

`\proofgraph[⟨options⟩]` Includes the rendered graph image at this point, passing the optional `⟨options⟩` to the underlying `\includegraphics` (e.g., `\proofgraph[width=\linewidth]`). This is the only feature that needs

`graphicx`, which the package therefore does not load on its own: a document that uses `\proofgraph` must load `graphicx` itself (loading `tikz` for the `hyperlinks` overlay also suffices). With the `autorun` option, `Graphviz` is run at the end of a compilation in which the graph changed (shell-escape required) to produce that image, so `\proofgraph` embeds the graph from the previous run and is up to date after two runs. Without `autorun`, no `Graphviz` is run: `\proofgraph` embeds a pre-existing `\jobname-proofgraph.<format>` file if you have rendered one under that name, and otherwise warns. When `hyperref` and `TikZ` are both loaded (with `hyperlinks` on and the `-Tplain` file from `autorun` present), every node becomes an internal hyperlink to the result it represents: `Graphviz` writes the node positions to that `-Tplain` file and `\proofgraph` overlays a transparent `\hyperref` area on each node (needed because the link annotations `Graphviz` puts in the `.pdf` are discarded by `\includegraphics`).

3.6 Options

selfloops Either `remove` (default), dropping edges from a result to itself, or `keep`, retaining them.

autorun Boolean (default false). Runs `Graphviz` automatically at the end of the run; requires compilation with shell-escape enabled. `Graphviz` is run only when the graph, the engine or the format has changed since the last rendering, so that repeated compilations converge and a build tool such as `latexmk` settles instead of rerunning indefinitely.

engine `Graphviz` layout engine (default `dot`); any `Graphviz` engine works, such as `neato`, `fdp`, `sfdp`, `circo` or `twopi` (see <https://graphviz.org/docs/layouts/>).

format Output format for `autorun` and `\proofgraph` (default `pdf`); any `Graphviz` output format works, such as `png`, `svg` or `ps` (see <https://graphviz.org/docs/outputs/>).

hyperlinks Boolean (default true). Makes the nodes of an embedded graph clickable when `hyperref` and `TikZ` are available (see `\proofgraph`); set it false to embed the graph as a plain image.

direction Either `usedby` (default), orienting an edge from the result that is used to the result that uses it (so an arrow `a->b` reads “a is used by b”), or `uses`, the reverse.

rankdir `Graphviz` graph direction (default `LR`).

cite Boolean (default false). Also captures `\cite` inside proofs as dependencies on external work, drawn as distinct nodes (Section 2), and, unless `statements` is off, in the body of a result too. A `\cite` note is kept: `\cite[Thm~3]{key}` labels the node “Thm 3, ...”. In the two-optional-argument form of `natbib` and `biblatex`, `\cite[see][Thm~3]{key}`, the post-note likewise labels the node and the pre-note is ignored. Every citation command of `LATEX`, `natbib` and `biblatex` that cites a work is captured the same way, whenever the document defines it, starred variants included:

`\citet`, `\citep`, `\citealt`, `\citealp`, `\parencite`, `\textcite`, `\autocite`, `\footcite`, `\footcitetext`, `\smartcite`, `\supercite`, `\fullcite` and `\footfullcite`, together with the commands printing only a fragment of a citation (`\citeauthor`, `\citefullauthor`, `\citetitle`, `\citeyear`, `\citeyearpar`, `\citedate`, `\citeurl`, `\citenum`) and all their capitalised forms. A work invoked twice in one proof, by `\citeauthor` and by `\citesay`, still yields a single edge. `\proofgraphcitecommand` adds any further command; `\usescite` and `\proofgraphciteedge` record a citation node by hand, with or without this option.

statements Boolean (default true). Captures cross-references, and citations when `\cite` is on, in the *body* of a result as well, not only in its proof and in its optional title: a statement reading “Under the hypotheses of Theorem 2...” or “... the bound of [?, Thm 3]” records that dependency on the result being stated, as a proof making the same reference would. The capture stops at the `\end` of the result, so references in the surrounding text record nothing. Set it false for a graph built from proofs alone, which is the right reading when statements refer to earlier results to contrast with them rather than to rely on them.

citelabel Either `key` (default), labelling citation nodes by their BibTeX key, or `tag`, using the printed citation tag instead (“[1]”, “[Knu74]”, “[Abiteboul et al. 1995]” ...), which is only known after a first run (so it needs two runs). It supports the standard, `hyperref`-wrapped and `natbib` (numeric and author–year) bibliographies. `biblatex` is supported in part: the tag is recovered for its numeric styles ([1]) and its alphabetic ones ([Knu68]), but not for its author–year styles, whose tag is a formatted name list that `biblatex` exposes as no single field. Whenever no clean tag can be recovered the node keeps the key, which is always a correct, if terser, label.

The note of a `\cite` is set where a citation prints it, inside the tag and after it: `\cite[pp.~23--27]{knuth}` gives a node labelled “[Knu68, pp. 23–27]”, carrying the dashes of the document, and “knuth, pp. 23–27” when the label is the key.

file Base name of the output file (default `\jobname`).

3.7 Use with `apxproof`

`apxproof` moves proofs to the appendix and typesets them there, so the references a proof makes are only executed at that later point. To attribute them to the right result, `proofgraph` relies on the hook `\apxproofhook`, which `apxproof` fires inside each deferred proof. This hook is available in `apxproof v1.4.1` and later; load the two packages in either order and no further setup is needed.

With an *earlier* `apxproof` the document still compiles cleanly and every result still becomes a node, but *no proof dependencies are captured for the proofs deferred to the appendix*, because `proofgraph` cannot tell which appendix proof belongs to which result (dependencies from any proofs left in the main text are still captured as usual). In that case, add the deferred ones manually with `\uses` (and `\proofof` to pin a proof to its result), or upgrade `apxproof`.

4 Limitations

- Environments declared through `\newtheorem` (or `\spnewtheorem`) *after* the package is loaded are tracked automatically. Environments predefined by the document class *before* that (as in `lipics`, `acmart` or the Springer classes) are also detected, provided their name is one of the common result names `proofgraph` probes at `\begin{document}` (`theorem`, `lemma`, `proposition`, `corollary`...); an environment with an unusual name predefined before the package is loaded must be added with `\proofgraphtrack`.
- Results must be labelled for references to them to be resolved.
- Two compilation runs are required, as for any cross-reference.

5 See also

`result-graph` [3] does for a Lean 4 formalisation what `proofgraph` does for an article: it draws the dependency graph of the results of a development, deliberately with the same vocabulary, node styles and edge convention, so that the graph of a formalisation and the graph of the paper it formalises look like one family. There the dependencies are not inferred from cross-references but read off the elaborated proof terms, and are therefore exact; they also distinguish, for free, a result used in the *statement* of another from one used only in its *proof*, which `proofgraph`, under the `statements` option, records but draws alike. And since a whole development is far too large for one figure, it is sliced into a small neighbourhood of each result, rather than drawn as a single graph as here.

References

- [1] Euclid. *Elements*, Book IX. c. 300 BCE.
- [2] A. Amarilli, F. Capelli, M. Monet, and P. Senellart. Connecting knowledge compilation classes and width parameters. *Theory of Computing Systems*, 64(5):861–914, 2020. doi:10.1007/s00224-019-09930-2.
- [3] P. Senellart. *result-graph: dependency graphs of the results of a Lean 4 project*. <https://github.com/PierreSenellart/result-graph>.

6 Implementation

The format required is that of October 2020, the first to provide `\NewDocumentCommand` in the kernel; `expl3`, which the package also uses, has been preloaded since the February 2020 release.

```

1 (*package)
2 \NeedsTeXFormat{LaTeX2e}[2020/10/01]
3 \ProvidesPackage{proofgraph}
4 [2026/07/31 v1.1.0 Dependency graph of results]

```

6.1 Dependencies

`amsthm` (loaded for its heading hooks) defines the `proof` environment with `\newenvironment`, which aborts with “Command `\proof` already defined” under a class that already provides one (e.g. the Springer `svjour3` `\proof`). When we are about to load `amsthm` ourselves (it is not already loaded), we save the class’s `proof`/`\endproof` and free them so `amsthm` loads cleanly, then restore the class’s versions afterwards, so its proof styling is kept (and `proofgraph` still hooks that `proof` environment). If `amsthm` is already loaded, `\RequirePackage` is a no-op and `\proof` is left untouched.

```

5 \ifpackageloaded{amsthm}{}{%
6   \ifundefined{proof}{}{%
7     \let\pgph@class@proof\proof \let\pgph@class@endproof\endproof}%
8     \let\proof\relax \let\endproof\relax}
9 \RequirePackage{amsthm}
10 \ifundefined{pgph@class@proof}{}{%
11   \let\proof\pgph@class@proof \let\endproof\pgph@class@endproof}
12 \RequirePackage{etoolbox}
13 \RequirePackage{xstring}
14 \RequirePackage{kvoptions}
15 \RequirePackage{pdftexcmds}

```

`pdftexcmds` provides `\pdf@filemdfivesum`, with which `autorun` recognises an unchanged graph and skips the `Graphviz` run (see `\pgph@maybrender`). On an engine that offers no file checksum the package still works, only without that optimisation.

`graphicx` is needed only to embed a rendered graph with `\proofgraph`; the core `.dot`-writing machinery does not use it. Rather than burden every document with it, we leave it unloaded and check for `\includegraphics` at the point of use (see `\proofgraph`). When the `hyperlinks` overlay is active `tikz` is loaded, which pulls `graphicx` in anyway.

`\pgph@trimsp` An expandable space-trimmer (labels inside a `\cite`/`\cref`/`\uses` list arrive with the space after each comma), built on the `expl3` primitive so it is robust to leading spaces and braces.

```

16 \ExplSyntaxOn
17 \cs_new:Npn \pgph@trimsp #1 { \tl_trim_spaces:n {#1} }
18 \ExplSyntaxOff

```

6.2 Options

```

19 \SetupKeyvalOptions{family=pgph,prefix=pgph@}
20 \DeclareStringOption[remove]{selfloops}

```

```

21 \DeclareBoolOption{autorun}
22 \DeclareStringOption[dot]{engine}
23 \DeclareStringOption[pdf]{format}
24 \DeclareStringOption[LR]{rankdir}
25 \DeclareStringOption[usedby]{direction}
26 \DeclareBoolOption{cite}
27 \DeclareStringOption[key]{citelabel}
28 \DeclareBoolOption[true]{statements}
29 \DeclareBoolOption[true]{hyperlinks}
30 \DeclareStringOption{file}
31 \ProcessKeyvalOptions*
32 \ifx\pgph@file\@empty\def\pgph@file{\jobname}\fi

```

6.3 Literal braces

We need catcode-12 braces to write the Graphviz digraph `{ }` block.

```

33 \begingroup
34   \catcode'\ [=1 \catcode'\ ]=2
35   \catcode'\ {=12 \catcode'\ }=12
36   \gdef\pgph@ob[{}]%
37   \gdef\pgph@cb[{}]%
38 \endgroup

```

6.4 Data structures

A global counter gives each result instance a unique numeric id. Nodes and edges are accumulated in list macros and emitted at end of document, so that editing commands (ignore, exclude, self-loops) can be applied to the complete graph regardless of where they appear.

```

39 \newcount\pgph@idcnt
40 \newcount\pgph@proofcnt
41 \newcount\pgph@citeslotcnt
42 \def\pgph@nodes{}
43 \def\pgph@edges{}
44 \def\pgph@ignores{}
45 \def\pgph@excludes{}
46 \let\pgph@curresult\@empty
47 \let\pgph@curproof\@empty

```

`\pgph@auxmap` The label-to-id map. `\label` inside a result records, both in memory and in the `.aux` file, that the user label maps to the current node id, so references resolve independently of `hyperref`.

```

48 \newcommand\pgph@auxmap[2]{\global\@namedef{pgph@map@#1}{#2}%
49   \ifcsname pgph@rmap@#2\endcsname\else
50     \global\@namedef{pgph@rmap@#2}{#1}\fi}
51 \newcommand\pgph@setmap[2]{%
52   \edef\pgph@tmpid{#2}%
53   \global\expandafter\edef\csname pgph@map@#1\endcsname{\pgph@tmpid}%
54   \ifcsname pgph@rmap@\pgph@tmpid\endcsname\else
55     \global\expandafter\def\csname pgph@rmap@\pgph@tmpid\endcsname{#1}%
56   \fi
57   \protected@write\@auxout{}{\string\pgph@auxmap{#1}{\pgph@tmpid}}%
58 }

```

6.5 Registering result environments

`\pgph@register` A result environment is tracked by installing a begin-hook that records a node. In every declaration command (`\newtheorem`, `\spnewtheorem`) the environment *name* is the first mandatory argument, so registration does not need to parse the remaining (varied) arguments. Each name met is queued as a *candidate* (deduplicated); the begin-hooks themselves are installed only at `\begin{document}`, once the inclusion and exclusion lists are settled, so the customisation commands below may appear anywhere in the preamble.

```

59 \let\pgph@candidates\@empty
60 \newcommand\pgph@register[1]{%
61   \edef\pgph@thisname{#1}%
62   \IfBeginWith\pgph@thisname{axp@}{-}{%
63     \ifcsname pgph@cand@\pgph@thisname\endcsname\else
64       \global\@namedef{pgph@cand@\pgph@thisname}{-}%
65       \ifx\pgph@candidates\@empty
66         \global\let\pgph@candidates\pgph@thisname
67       \else
68         \xdef\pgph@candidates{\pgph@candidates,\pgph@thisname}%
69       \fi
70     \fi
71   }%
72 }
```

We wrap `\newtheorem` (and, for Springer classes, `\spnewtheorem`) to register each newly declared environment, replaying the original command so its own argument parsing is untouched. The starred (unnumbered) forms are *not* auto-registered: a results graph wants numbered statements, and these forms are commonly used for unnumbered repeats of a result (e.g., the appendix copies produced by `apxproof` and `myproofs`), which would otherwise appear as spurious unnumbered duplicate nodes. Use `\proofgraphtrack` to track an unnumbered environment deliberately.

```

73 \let\pgph@orig@newtheorem\newtheorem
74 \renewcommand\newtheorem{\@ifstar{\pgph@nt@star}{\pgph@nt@plain}}
75 \newcommand\pgph@nt@plain[1]{%
76   \pgph@register{#1}\pgph@orig@newtheorem{#1}}
77 \newcommand\pgph@nt@star[1]{\pgph@orig@newtheorem*{#1}}
78 \ifdefined\spnewtheorem
79   \let\pgph@orig@spnewtheorem\spnewtheorem
80   \renewcommand\spnewtheorem{%
81     \@ifstar{\pgph@spnt@star}{\pgph@spnt@plain}}
82   \newcommand\pgph@spnt@plain[1]{%
83     \pgph@register{#1}\pgph@orig@spnewtheorem{#1}}
84   \newcommand\pgph@spnt@star[1]{\pgph@orig@spnewtheorem*{#1}}
85 \fi
```

`\proofgraphtrack`
`\proofgraphuntrack`
`\proofgraphtrack`
`\proofgraphuntrack`

Result environments are detected automatically, but the analysis can be tuned. `\proofgraphtrack{⟨names⟩}` forces the listed environments to be tracked, overriding any exclusion; `\proofgraphuntrack{⟨names⟩}` excludes the listed environment *types*. Expository environments are excluded by default: `definition`, `example`, `remark`, `note`. Both commands belong in the

preamble. `\pgph@defaultenvs` is the list of class-predefined names probed at `\begin{document}`; `\pgph@defaultexclude` seeds the exclusion set.

```

86 \newcommand\pgph@defaultenvs{theorem,lemma,proposition,corollary,%
87   definition,conjecture,claim,fact,remark,example,observation,%
88   notation,problem,question,property,assumption,hypothesis,principle}
89 \newcommand\pgph@defaultexclude{definition,example,remark,note}
90 \newcommand\proofgraphtrack[1]{%
91   \@for\pgph@one:=#1\do{%
92     \edef\pgph@one{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
93     \csundef{pgph@xtype@\pgph@one}%
94     \global\@namedef{pgph@force@\pgph@one}{}%
95     \expandafter\pgph@register\expandafter{\pgph@one}%
96   }%
97 }
98 \newcommand\proofgraphuntrack[1]{%
99   \@for\pgph@one:=#1\do{%
100     \edef\pgph@one{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
101     \csundef{pgph@force@\pgph@one}%
102     \global\@namedef{pgph@xtype@\pgph@one}{}%
103   }%
104 }
105 \expandafter\proofgraphuntrack\expandafter{\pgph@defaultexclude}

```

Environments predefined by the document class (acmart, lipics, Springer classes) are declared before the package is loaded, so the wrappers above never see them. At `\begin{document}` we add every common result name that exists to the candidate list, then install a begin-hook for each candidate that is forced or not excluded.

```

106 \newcommand\pgph@dohook[1]{%
107   \ifcsname pgph@hooked@#1\endcsname\else
108     \global\@namedef{pgph@hooked@#1}{}%
109     \AtBeginEnvironment{#1}{\pgph@beginresult{#1}}%
110   \fi
111 }
112 \newcommand\pgph@maybehook[1]{%
113   \ifcsname pgph@force@#1\endcsname
114     \pgph@dohook{#1}%
115   \else
116     \ifcsname pgph@xtype@#1\endcsname\else\pgph@dohook{#1}\fi
117   \fi
118 }
119 \newcommand\pgph@resolveregistration[1]{%
120   \@for\pgph@one:=#1\do{%
121     \ifcsname\pgph@one\endcsname%
122       \expandafter\pgph@register\expandafter{\pgph@one}\fi%
123   \ifx\pgph@candidates\@empty\else
124     \@for\pgph@one:=\pgph@candidates\do{%
125       \expandafter\pgph@maybehook\expandafter{\pgph@one}}%
126   \fi
127 }
128 \AtBeginDocument{%
129   \expandafter\pgph@resolveregistration\expandafter{\pgph@defaultenvs}

```

`\pgph@beginresult` Start a result: allocate an id and record the node. The node is keyed and styled by `\pgph@hooknames`
`\pgph@recordresult`

the environment name (third argument, also a fallback label), but displayed by the result's *formatted* name ("Theorem", "Lemma"), captured from the heading. We get that name by locally wrapping the heading macros, whose first two arguments are the formatted name and the number. Which macro carries the *optional note* (where a restatement's cross-reference lives) depends on the setup: `amsthm`, which `proofgraph` always loads, routes every heading through `\@begintheorem`, defined as `#1#2[#3]` with the note as the bracketed `#3` (and leaves `\@opargbegintheorem` `\relax`); the L^AT_EX kernel instead uses a two-argument `\@begintheorem` with a separate three-argument `\@opargbegintheorem` for the note; the Springer classes use `\@spbegintheorem/\@spopargbegintheorem`. We detect the `amsthm` form (`\@opargbegintheorem` is `\relax`) and read the bracketed note there, otherwise we hook `\@opargbegintheorem`; either way the note goes to `\pgph@scanrefs`. These redefinitions are confined to the current environment group, so they revert at `\end`. We also locally redefine `\label` to populate the map.

The note read as a *delimited* argument is handed back to the original macro inside braces, `[{\note}]` rather than `[<note>]`. TeX strips the outer braces of a delimited argument that is one brace group in its entirety, so a title written `[{\cite[p.~37]{a-1}}]` – the bracing the `amsthm` manual prescribes to hide the `]` of the citation note – would otherwise reach the original macro unbraced, whose own scan for `]` would then stop inside the citation and take the rest of the note for text. Rebracing restores exactly what was written: the added braces are stripped again by that scan.

Nodes are accumulated in *reverse* order of appearance (`\pgph@prependnode`). With `rankdir=LR`, `Graphviz` stacks disconnected components vertically in the order they are read, from the bottom up; emitting the results last-to-first therefore lays them out top-to-bottom in document order (first result on top), while connected results stay grouped. This is a layout tendency, not a guarantee, but needs no `pack` trickery.

Unless `statements` is switched off, the *body* of the result is treated like the body of a proof of that result (`\pgph@beginstatement`), so that a cross-reference or a citation made in the statement itself becomes a dependency. Those assignments are local to the environment group, as the ones above are, so they revert at `\end`.

A result opened *inside* a proof – a claim stated in the course of an argument, and proved there – is a step of that proof, so `\pgph@supportededge` records the proof as depending on it: the claim would otherwise be drawn with whatever its own proof uses hanging below it and nothing above, as a component detached from the result it was stated to establish. The edge is recorded against the enclosing proof rather than a result, so it follows the same chain of owners as any other, `\proofof` and nesting included, and it passes through the ordinary filters, so `\proofgraphignore` drops it like any other. Nothing is recorded for a claim the document leaves unnumbered: `\pgph@node` excludes such a result, and the edge to it goes with it.

```

130 \newcommand\pgph@supportededge{%
131   \ifx\pgph@encproof\@empty\else
132     \pgph@addedge{\pgph@encproof}{node:\pgph@curresult}%
133   \fi
134 }
135 \newcommand\pgph@prependnode[1]{%
136   \xdef\pgph@nodes{\unexpanded{#1}\unexpanded\expandafter{\pgph@nodes}}%
137 }

```

```

138 \newcommand\pgph@beginresult[1]{%
139   \let\pgph@encproof\pgph@curproof
140   \global\advance\pgph@idcnt\@ne
141   \xdef\pgph@curresult{\the\pgph@idcnt}%
142   \protected@edef\pgph@tmp{%
143     \noexpand\pgph@node{\pgph@curresult}{#1}{#1}}%
144   \expandafter\pgph@prependnode\expandafter{\pgph@tmp}%
145   \pgph@supportedge
146   \pgph@hooknames
147   \ifx\label\pgph@maplabel\else\let\pgph@savedlabel\label\fi
148   \let\label\pgph@maplabel
149   \ifpgph@statements\pgph@beginstatement\fi
150 }
151 \newcommand\pgph@hooknames{%
152   \ifdefined\@begintheorem
153     \let\pgph@orig@bt\@begintheorem
154     \ifx\@opargbegintheorem\relax
155       \def\@begintheorem##1##2[##3]{%
156         \pgph@recordresult{##1}{##2}\pgph@scanrefs{##3}%
157         \pgph@orig@bt{##1}{##2}[##3]}%
158     \else
159       \def\@begintheorem##1##2{%
160         \pgph@recordresult{##1}{##2}\pgph@orig@bt{##1}{##2}}%
161     \fi
162   \fi
163   \ifdefined\@opargbegintheorem\ifx\@opargbegintheorem\relax\else
164     \let\pgph@orig@obt\@opargbegintheorem
165     \def\@opargbegintheorem##1##2##3{%
166       \pgph@recordresult{##1}{##2}\pgph@scanrefs{##3}%
167       \pgph@orig@obt{##1}{##2}{##3}}%
168   \fi\fi
169   \ifdefined\@spbegintheorem
170     \let\pgph@orig@spbt\@spbegintheorem
171     \def\@spbegintheorem##1##2##3##4{%
172       \pgph@recordresult{##1}{##2}\pgph@orig@spbt{##1}{##2}{##3}{##4}}%
173   \fi
174   \ifdefined\@spopargbegintheorem
175     \let\pgph@orig@spobt\@spopargbegintheorem
176     \def\@spopargbegintheorem##1##2##3##4##5{%
177       \pgph@recordresult{##1}{##2}\pgph@scanrefs{##3}%
178       \pgph@orig@spobt{##1}{##2}{##3}{##4}{##5}}%
179   \fi
180 }

```

`\pgph@scanrefs` A result whose optional title carries a cross-reference, e.g., a `result` whose title is `[(Corollary~\ref{cor:x})]`, is understood to restate or build on that result: we record an edge from the current result to each label referenced in the title. We capture these by typesetting the title once into a discarded box with the capturing commands installed and `\pgph@curproof` pointing at the current result. Everything happens inside a group and `\pgph@curproof` is only *read* while the box is built, so a local assignment suffices and nothing outside the title is affected; a global one would escape the enclosing environment, which under `statements` (where `\pgph@curproof` is already set on entering the result) would leave every

subsequent reference attributed to that result.

```

181 \newcommand\pgph@scanrefs[1]{%
182   \ifx\pgph@curresult\@empty\else
183     \begingroup
184       \let\pgph@curproof\pgph@curresult
185       \pgph@installcapture
186       \setbox\z@\hbox{#1}%
187     \endgroup
188   \fi
189 }
```

The formatted name is the first argument of every heading macro (`\@begintheorem/\@opargbegintheorem` for `amsthm/\newtheorem`, `\@spbegintheorem/\@spopargbegintheorem` for Springer's `\spnewtheorem`); the second argument is the number. We use that second argument only to tell *whether* the result is numbered (it is empty for the starred, unnumbered forms): an unnumbered result records no number, so it is dropped at emission unless tracked explicitly. The number *value*, when present, is read from `\@currentlabel` rather than from that argument, which some classes pollute (e.g., `lipics` injects `\advance\par@deathcycles\@ne`). `\@currentlabel`, set by the result's `\refstepcounter` just before the heading, is the clean number and needs no `\label`. Values are reduced to plain strings now, with fragile wrappers neutralized (notably `apxproof`'s forward-link). The first heading wins (guarded on the name), ignoring nested headings. The emptiness of the number argument is tested without expanding it, so a polluted (but non-empty) number does not trip the test.

```

190 \newcommand\pgph@recordresult[2]{%
191   \ifcsname pgph@name@\pgph@curresult\endcsname\else
192     \begingroup
193       \let\protect\@empty
194       \def\axp@forward@link##1##2{##2}%
195       \edef\pgph@tmpname{#1}%
196       \global\expandafter\let\csname pgph@name@\pgph@curresult\endcsname
197         \pgph@tmpname
198       \global\@namedef{pgph@seen@\pgph@curresult}{}%
199       \def\pgph@tmpn{#2}%
200       \ifx\pgph@tmpn\@empty\else
201         \edef\pgph@tmpnum{\@currentlabel}%
202         \global\expandafter\let\csname pgph@num@\pgph@curresult%
203           \endcsname\pgph@tmpnum
204       \fi
205     \endgroup
206   \fi
207 }
```

`\pgph@maplabel` records the label-to-node mapping. As a fallback for classes whose headings pass through none of the hooked heading macros (so no heading was `seen`), it also captures the number from `\@currentlabel`. We do *not* do this once a heading was seen: there, an absent number means the result is genuinely unnumbered (a starred form), and `\@currentlabel` could be a stale value from an earlier result. The value is expanded to a plain string *now*, inside a group where fragile wrappers are neutralized, so that nothing danger-

ous survives to the emission pass: in particular, under `apxproof` forward-linking, `\@currentlabel` holds `\axp@forward@link{<target>}{<number>}`, whose hyper-link machinery must never be expanded in a non-typesetting context (it would run away). Reducing it to its number argument keeps it harmless.

```
\pgph@maplabel
208 \newcommand\pgph@maplabel[1]{%
209   \pgph@setmap{#1}{\pgph@curresult}%
210   \ifcsname pgph@num@\pgph@curresult\endcsname\else
211   \ifcsname pgph@seen@\pgph@curresult\endcsname\else
212     \begingroup
213       \let\protect\@empty
214       \def\axp@forward@link##1##2{##2}%
215       \edef\pgph@tmpnum{\@currentlabel}%
216       \global\expandafter\let\csname pgph@num@\pgph@curresult%
217         \endcsname\pgph@tmpnum
218     \endgroup
219   \fi
220 \fi
221 \pgph@savetlabel{#1}%
222 }
```

6.6 Hooking proofs

Inside a proof we redefine the reference commands to capture edges from the proved result.

```
223 \newcommand\pgph@adddedge[2]{%
224   \protected@edef\pgph@tmpedge{\noexpand\pgph@edge{#1}{#2}}%
225   \expandafter\g@addto@macro\expandafter\pgph@edges%
226     \expandafter{\pgph@tmpedge}%
227 }
228 \newcommand\pgph@recordref[1]{%
229   \ifpgph@nocapture\else
230   \ifx\pgph@curproof\@empty\else
231     \@for\pgph@one:=#1\do{%
232       \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
233       \pgph@adddedge{\pgph@curproof}{\pgph@k}}%
234   \fi\fi
235 }
```

`\ifpgph@nocapture` The capture is installed for the whole of a proof, and a proof long enough to break the page has the page furniture built while it is open. The running head is set from the marks, so a cross-reference or a citation in a sectional title is *executed again* there, in the output routine, and would be recorded as a dependency of the proof the break fell in: a document whose section title names a result gained an edge from that result to whichever result the proof belonged to. Recording is therefore suspended while the page is assembled.

The kernel neutralizes `\label`, `\index` and `\glossary` at the same point, in `\@outputpage`, and offers the hook `build/page/reset` there, inside the group that encloses the shipped box, so a local switch set from the hook covers the head and foot and is undone by itself. Where the hook is not declared (a kernel older than the hook, a class with an output routine of its own that skips it), `\@outputpage`

is wrapped instead, which needs the switch to be set globally as the group ends inside the box being shipped.

```

236 \newif\ifpgph@nocapture
237 \newcommand\pgph@wrapoutputpage{%
238   \let\pgph@orig@outputpage\@outputpage
239   \def\@outputpage{%
240     \global\pgph@nocapturetrue
241     \pgph@orig@outputpage
242     \global\pgph@nocapturefalse}%
243 }
244 \AtBeginDocument{%
245   \ifdefined\IfHookExistsTF
246     \IfHookExistsTF{build/page/reset}%
247     {\AddToHook{build/page/reset}{\pgph@nocapturetrue}}%
248     {\pgph@wrapoutputpage}%
249   \else
250     \pgph@wrapoutputpage
251   \fi
252 }

```

`\pgph@beginproof` Attach the proof to the current result and install the capturing versions of the reference commands (only those that exist).

The proof is not recorded as the result it proves but as a *proof instance* of its own, $p\langle n \rangle$, whose owning result is held in `\pgph@powner@p<n>` and read only at emission. An edge captured in the proof therefore does not fix its source when it is seen, which is what lets `\proofof` pin the whole proof and not merely the references that follow it. Were the source fixed at capture, a proof opening with “By Theorem 1” and naming its own result only at the end would give that first reference to whichever result happens to precede the proof. The indirection also lets a proof be pinned to a result labelled later in the document, the label being resolved with all the others when the graph is written.

The capturing commands are `\protected` so that, when a reference appears in a moving argument inside a proof (e.g., a `\caption`), they do not expand into the `.aux/.lof` and leak internal tokens.

```

253 \protected\def\pgph@cap@ref#1{\pgph@recordref{#1}\pgph@saved@ref{#1}}
254 \protected\def\pgph@cap@cref#1{\pgph@recordref{#1}\pgph@saved@cref{#1}}
255 \protected\def\pgph@cap@Cref#1{\pgph@recordref{#1}\pgph@saved@Cref{#1}}
256 \protected\def\pgph@cap@autoref#1{%
257   \pgph@recordref{#1}\pgph@saved@autoref{#1}}
258 \protected\def\pgph@cap@eqref#1{%
259   \pgph@recordref{#1}\pgph@saved@eqref{#1}}
260 \protected\def\pgph@cap@vref#1{\pgph@recordref{#1}\pgph@saved@vref{#1}}

```

`zref-clever`’s `\zcref` is the one captured command that is not simply $\langle cs \rangle \{ \langle labels \rangle \}$: it is `\zcref*[\langle options \rangle] \{ \langle labels \rangle \}`, and both the star and the options have to be handed on unchanged. Its mandatory argument is a comma-separated list, which `\pgph@recordref` already splits.

```

261 \NewDocumentCommand\pgph@cap@zcref{s0{m}}{%
262   \pgph@recordref{#3}%
263   \IfBooleanTF{#1}{\pgph@saved@zcref*{#2}{#3}}{\pgph@saved@zcref{#2}{#3}}%
264 }
265 \newcommand\pgph@install[1]{%

```

```

266 \ifcsname #1\endcsname
267   \expandafter\ifx\csname #1%
268     \expandafter\endcsname\csname pgph@cap@#1\endcsname
269   \else
270     \expandafter\let\csname pgph@saved@#1%
271       \expandafter\endcsname\csname #1\endcsname
272     \expandafter\let\csname #1%
273       \expandafter\endcsname\csname pgph@cap@#1\endcsname
274   \fi
275 \fi
276 }

```

The set of captured commands is needed in three places (a proof, the optional title of a result, and an `apxproof` deferred proof), so it is named once here: adding a command to the list must not be a matter of remembering three call sites.

```

277 \newcommand\pgph@installrefs{%
278   \pgph@install{ref}\pgph@install{cref}\pgph@install{Cref}%
279   \pgph@install{autoref}\pgph@install{eqref}\pgph@install{vref}%
280   \pgph@install{zceref}%
281 }

```

The cross-reference commands and, when the `cite` option is on, the citation commands are always installed together, so the pair has a name of its own: every place that opens a capturing context (a proof, the optional title of a result, and, with `statements`, the body of a result) installs exactly the same set.

```

282 \newcommand\pgph@installcapture{%
283   \pgph@installrefs
284   \ifpgph@cite\pgph@installcite\fi
285 }

```

A proof may sit inside another one, either because the proof of a claim stated in the course of a proof is itself a `proof` environment, or because the document writes structured proofs, whose steps are proofs nested in the proof of the whole (as `pf2` does). The two cases want different owners, and what tells them apart is whether a *result* has been opened since the enclosing context began: the proof of a claim follows the claim, whereas the step of a structured proof follows nothing. So each capturing context records, in `\pgph@pres@{ctx}`, the result current when it began; a proof opening while that result is still the current one is a *sub-proof* and takes the enclosing context as its owner, and any other proof takes the current result, as before.

Owning the enclosing context rather than a result is what lets a `\proofof` in an outer proof reach the inner ones: the pin is read where the chain of owners is followed, at emission, so it applies to a nested reference just as it does to one made directly in the pinned proof. An inner `\proofof` still pins its own proof alone, which is what the detached proof of a claim needs.

```

286 \newcommand\pgph@setpres[1]{%
287   \global\expandafter\let\csname pgph@pres@#1\endcsname\pgph@curresult
288 }
289 \newcommand\pgph@setpowner[2]{%
290   \global\expandafter\let\csname pgph@powner@#1\endcsname#2%
291 }
292 \newcommand\pgph@beginproof{%
293   \global\advance\pgph@proofcnt\@ne
294   \edef\pgph@newproof{p\the\pgph@proofcnt}%

```

```

295 \pgph@setpres\pgph@newproof
296 \edef\pgph@ncur{\pgph@curresult}%
297 \edef\pgph@npres{%
298   \ifcsname pgph@pres@\pgph@curproof\endcsname
299     \csname pgph@pres@\pgph@curproof\endcsname
300   \else\pgph@nomatch\fi}%
301 \ifx\pgph@ncur\pgph@npres
302   \pgph@setpowner\pgph@newproof\pgph@curproof
303 \else
304   \pgph@setpowner\pgph@newproof\pgph@curresult
305 \fi
306 \let\pgph@curproof\pgph@newproof
307 \pgph@installcapture
308 \pgph@hookproofhead
309 }

```

The sentinel stands for “no enclosing context”, and must be something no result id can be, so that the comparison fails rather than matching an empty `\pgph@curresult` outside any result.

```

310 \newcommand\pgph@nomatch{-}

```

`\pgph@beginstatement` Under the `statements` option the body of a result captures like the body of its proof. The proof-heading hook is deliberately *not* installed here: it turns a cross-reference in an optional title into a `\proofof`, which is right for a proof titled “of Theorem 1” but not for a result titled “(restates Theorem 1)”, where the reference is an ordinary dependency.

```

311 \newcommand\pgph@beginstatement{%
312   \let\pgph@curproof\pgph@curresult
313   \pgph@setpres\pgph@curproof
314   \pgph@installcapture
315 }

```

When the `proof` environment is itself a theorem-like environment (as with Springer’s `\spnewtheorem*{proof}`), its optional title is set by `\@opargbegintheorem/\@spopargbegintheorem`. We wrap these for the duration of the proof so that a title such as [of Theorem~\ref{thm:x}] sets `\proofof{thm:x}`, attributing the proof to its result. As in `\pgph@hooknames`, a `\relax \@opargbegintheorem` is left alone: that is the `amsthm` setup, where the macro is never called, and giving it a meaning would make `\pgph@hooknames` read a result nested in the proof as having the kernel’s two-argument heading rather than `amsthm`’s.

```

316 \newcommand\pgph@hookproofhead{%
317   \ifdefined\@opargbegintheorem\ifx\@opargbegintheorem\relax\else
318     \let\pgph@orig@obtp\@opargbegintheorem
319     \def\@opargbegintheorem##1##2##3{%
320       \pgph@scanproofarg{##3}\pgph@orig@obtp{##1}{##2}{##3}}%
321   \fi\fi
322   \ifdefined\@spopargbegintheorem
323     \let\pgph@orig@spobtp\@spopargbegintheorem
324     \def\@spopargbegintheorem##1##2##3##4##5{%
325       \pgph@scanproofarg{##3}%
326       \pgph@orig@spobtp{##1}{##2}{##3}{##4}{##5}}%
327   \fi

```

```

328 \ifdefined\@Opargbegintheorem
329   \let\pgph@orig@Otp\@Opargbegintheorem
330   \def\@Opargbegintheorem##1##2##3##4{%
331     \pgph@scanproofarg{##2}\pgph@orig@Otp{##1}{##2}{##3}{##4}}%
332   \fi
333 }

```

`\pgph@scanproofarg` A proof whose optional title names its result (a detached proof) has its result given by that title, e.g., `\thm:x` for an optional argument [of `Theorem~\ref{thm:x}`], rather than by the most recently opened result. We capture this by wrapping the `proof` environment so that, when an optional title is given, the cross-reference it contains is turned into a `\proofof` (the title is typeset once into a discarded box with `\ref` and friends redefined). Without an optional title the original environment is used unchanged.

```

334 \NewDocumentCommand\pgph@zcpproofof{s0{m}}{\proofof{#3}}
335 \newcommand\pgph@scanproofarg[1]{%
336   \begingroup
337     \let\protect\@empty
338     \def\ref##1{\proofof{##1}}\def\cref##1{\proofof{##1}}%
339     \def\Cref##1{\proofof{##1}}\def\autoref##1{\proofof{##1}}%
340     \let\zcref\pgph@zcpproofof
341     \setbox\z@\hbox{#1}%
342   \endgroup
343 }

```

To capture the optional title of a detached `amsthm` proof (an optional [of `Theorem~\ref{thm:x}`]) we redefine the outer `\proof` so that, when a title is given, it is scanned and then handed to the *saved* outer macro, which parses it as it always did. Delegating to the saved outer macro rather than to the inner one means we need to know neither the name of the inner macro nor the environment's default title, so the same wrapper serves any proof environment, whatever its default. No recursion occurs: the saved meaning is captured before the redefinition.

We must wrap *only* a genuine optional-argument environment: a package that captures the proof body verbatim (`apxproof`, `myproofs`) redefines `proof` with no optional argument, and wrapping it would break that capture. We recognise the optional-argument form by the `\@protected@testopt` in its definition. When the proof environment is instead a theorem-like environment (Springer `\spnewtheorem*{proof}`, or a `\newtheorem` of one's own) its title is already handled by `\pgph@hookproofhead`, so there we just install the begin-hook.

```

344 \newcommand\pgph@wrapproofone[1]{%
345   \@ifundefined{#1}{}{%
346     \edef\pgph@proofmeaning{\expandafter\meaning\csname #1\endcsname}%
347     \edef\pgph@testoptstr{\detokenize{\@protected@testopt}}%
348     \IfSubStr\pgph@proofmeaning\pgph@testoptstr
349       {\pgph@redefproof{#1}}%
350       {\AtBeginEnvironment{#1}{\pgph@beginproof}}%
351   }%
352 }
353 \newcommand\pgph@redefproof[1]{%
354   \expandafter\let\csname pgph@savenv@#1\expandafter\endcsname
355     \csname #1\endcsname
356   \expandafter\def\csname #1\endcsname{\pgph@beginproof

```

```

357 \ifnextchar[{\pgph@proofopt{#1}}%
358 {\csname pgph@savenv@#1\endcsname}}%
359 }
360 \def\pgph@proofopt#1[#2]{%
361 \pgph@scanproofarg{#2}\csname pgph@savenv@#1\endcsname[{#2}]}

```

`\proofgraphproofenv`

`\proofgraphproofenv` Which environments are proofs is a list, `proof` to begin with; `\pgph@proofenvs` `\proofgraphproofenv{<names>}` adds to it, for a document whose proofs live in an environment of another name (pf2's structured proofs under a name that avoids the clash with `amsthm`, a `\newtheorem{claimproof}` for the proofs of claims...). A proof is not a result, so the names are excluded from result tracking as well; `\proofgraphtrack` still overrides that for an environment one wants drawn as both.

```

362 \newcommand\pgph@proofenvs{proof}
363 \newcommand\proofgraphproofenv[1]{%
364 \g@addto@macro\pgph@proofenvs{,#1}%
365 \proofgraphtrack{#1}%
366 }

```

Packages such as `apxproof` capture the body of a `proof` verbatim (to defer it to the appendix) and replay it there. Patching the `proof` environment then corrupts that capture, and the references are only executed at replay time anyway. So we wrap `proof` only when no such package is active; otherwise the deferred-proof capture (below) takes over. The exemption is for `proof` alone: an environment added with `\proofgraphproofenv` is not one `apxproof` defers, so it is wrapped as usual. This happens at `\begin{document}`, once we know what is loaded and what the preamble has added to the list.

```

367 \newif\ifpgph@apxproof
368 \newcommand\pgph@wrapproofs{%
369 \for\pgph@one:=\pgph@proofenvs\do{%
370 \edef\pgph@pe{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
371 \ifboolexpr{ bool {pgph@apxproof} and test {\ifdefstring{\pgph@pe}{proof}} }%
372 {\expandafter\pgph@wrapproofone\expandafter{\pgph@pe}}%
373 }%
374 }
375 \AtBeginDocument{%
376 \ifpackageloaded{apxproof}{\pgph@apxprooftrue\pgph@apxinit}{}%
377 \pgph@wrapproofs
378 }

```

`\pgph@apxinit` Under `apxproof`, deferred proofs are typeset in the appendix, where their references finally execute. `apxproof` fires `\apxproofhook` inside each such proof, passing the `axp@r<n>` label of the proved theorem, which (because `apxproof` also attaches that label to the theorem in the main text) is already in our map. We resolve it to the source node and install the reference-capturing commands for the duration of the proof.

```

379 \newcommand\pgph@apxinit{%
380 \def\apxproofhook##1{\pgph@apxproofcapture{##1}}%
381 \newcommand\pgph@apxproofcapture[1]{%
382 \edef\pgph@curproof{\pgph@resolve{#1}}%
383 \ifx\pgph@curproof\@empty\else

```

```

384 \pgph@setpres\pgph@curproof
385 \pgph@installcapture
386 \fi
387 }

```

`\pgph@hookcite` Optional citation capture, applied to `\cite` and, when they are defined (`natbib`, `biblatex`), to `\citet` and `\citep` as well, all sharing one capture chain parameterized by the command name: external references become `cite:-`prefixed target labels, drawn later as distinct nodes. The node identity is the pair (key, note): a `\cite[⟨note⟩]{⟨key⟩}` inside a proof is distinguished from a `\cite` of the same key with a *different* note, so that citing two different results of the same work (say two theorems of one book) yields two nodes rather than one mislabelled with whichever note was seen first. Each pair is assigned a numeric *slot* (`\pgph@citeslot`); the edge target is `cite:⟨slot⟩`, and the slot remembers the key (for the tag and the bibliography hyperlink) and the note (for the label). The note is reduced to a plain string at capture time (fragile wrappers neutralized, `~` normalised to a space), which also serves as the signature so `[Thm 1]` and `[Thm~1]` coincide. We peek for the optional argument with `\@ifnextchar` rather than declaring one, so that a note-less `\cite{⟨key⟩}` is passed on *as written*: re-emitting it as `\cite[] {⟨key⟩}` would make the typeset citation grow a spurious empty note (“[1,]”). A second optional argument is peeked for in the same way, to support the two-argument form `\cite[⟨pre-note⟩][⟨note⟩]{⟨key⟩}` of `natbib` and `biblatex`; there the *second* argument is the note recorded for the node (matching the one-argument semantics, where the lone argument is the post-note), while the pre-note (“see”, “e.g.”) qualifies the act of citing rather than identifying the cited result, and is only replayed to the saved `\cite`. Without this peek, the mandatory-argument scan would grab the bare token `[` as the key and typeset the rest of the citation as text, giving confusing errors (“Missing \$ inserted”) deep in the proof body. A star (`\citet*`, `\citep*`: full author list) is likewise peeked for and replayed; it affects rendering only, not the recorded (key, note) pair. As in `\pgph@hooknames`, the notes are replayed braced, `[{⟨note⟩}]`, so that a note written `[{p.~37, [sic]}]` to hide a `]` reaches the saved command as written rather than cut short at that `]`. The replay head (saved command plus optional star) is frozen in `\pgph@replay` by the star-peek before the arguments are scanned. As in `\pgph@install`, installation is skipped (per command) when the command is already the capture macro; otherwise a proof nested inside another proof would save the capture as the saved command, and the first citation in the inner proof would recurse forever.

```

388 \newcommand\pgph@gxdefcs[2]{%
389   \global\expandafter\edef\csname#1\endcsname{#2}}
390 \newcommand\pgph@cap@citegen[1]{%
391   \def\pgph@citecmd{#1}%
392   \@ifstar{\pgph@cap@cite@x*}{\pgph@cap@cite@x{}}
393 \def\pgph@cap@cite@x#1{%
394   \edef\pgph@replay{%
395     \expandafter\noexpand\csname pgph@saved@\pgph@citecmd\endcsname#1}%
396   \@ifnextchar[\pgph@cap@cite@opt\pgph@cap@cite@noopt}
397 \def\pgph@cap@cite@opt[#1]{%
398   \@ifnextchar[{\pgph@cap@cite@optii{#1}}{\pgph@cap@cite@opti{#1}}}
399 \def\pgph@cap@cite@opti#1#2{%
400   \pgph@cap@citerec{#1}{#2}\pgph@replay[{#1}]{#2}}
401 \def\pgph@cap@cite@optii#1#2#3{%

```

```

402 \pgph@cap@citerec{#2}{#3}\pgph@replay[#{1}][#{2}][#{3}]
403 \def\pgph@cap@cite@noopt#1{\pgph@cap@citerec{#{1}}\pgph@replay{#{1}}}
404 \newcommand\pgph@cap@citerec[2]{%
405   \ifpgph@nocapture\else
406   \ifx\pgph@curproof\@empty\else
407     \@for\pgph@one:=#2\do{%
408       \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
409       \pgph@citeslot{\pgph@k}{#1}%
410       \pgph@addedge{\pgph@curproof}{cite:\pgph@curslot}}%
411   \fi\fi
412 }
413 \newcommand\pgph@citeslot[2]{%
414   \begingroup
415     \pgph@citesanitize
416     \xdef\pgph@gnotestr{#2}%
417   \endgroup
418   \edef\pgph@sig{#1@}\detokenize\expandafter{\pgph@gnotestr}}%
419   \ifcsname pgph@cslot@\pgph@sig\endcsname
420     \edef\pgph@curslot{\csname pgph@cslot@\pgph@sig\endcsname}%
421   \else
422     \global\advance\pgph@citeslotcnt\@ne
423     \edef\pgph@curslot{\the\pgph@citeslotcnt}%
424     \pgph@gxdefcs{pgph@cslot@\pgph@sig}{\pgph@curslot}%
425     \pgph@gxdefcs{pgph@cskey@\pgph@curslot}{#1}%
426     \ifx\pgph@gnotestr\@empty\else
427       \pgph@gxdefcs{pgph@citenote@\pgph@curslot}{\pgph@gnotestr}%
428     \fi
429   \fi
430 }

```

`\pgph@citecmds` Every citation command with the signature `\cs*[(pre)][(post)] {(keys)}` is captured by the same `\pgph@cap@citegen`, so which commands are captured `\proofgraphcitecommand` is a list rather than code. The default list holds the citation commands of L^AT_EX, natbib and biblatex that cite a work; `\proofgraphcitecommand` adds more. Naming a command that the document never defines costs nothing, as `\pgph@installciteone` installs only over an existing command, so the list may name every spelling a citation package might provide.

The commands printing only a *fragment* of a citation (`\citeauthor`, `\citeyear`...) are in the list as well. They commonly accompany a real citation of the same work, but a repetition is harmless: a (key, note) pair already seen reuses its slot, and `\pgph@writeedge` draws an edge once, so the second mention of a work adds nothing to the graph. Capturing them means that a proof invoking a work by author or by year alone, which does happen, is not silently left without its edge.

The biblatex *multicite* commands (`\cites`, `\parencites`, `\textcites`) cannot be captured at all: their `((pre))((post))` syntax and repeated key groups are a different grammar, which `\pgph@cap@citegen` does not parse.

```

431 \newcommand\pgph@citecmds{%
432   cite,Cite,citet,Citet,citep,Citep,%
433   citealt,Citealt,citealp,Citealp,%
434   parencite,Parencite,textcite,Textcite,autocite,Autocite,%
435   footcite,Footcite,footcitetext,smartcite,Smartcite,supercite,%
436   fullcite,footfullcite,%

```

```

437 citeauthor,Citeauthor,citefullauthor,citetitle,Citetitle,%
438 citeyear,Citeyear,citeyearpar,citedate,Citedate,citeurl,citenum}
439 \newcommand\proofgraphcitecommand[1]{\g@addto@macro\pgph@citecmds{,#1}}
440 \newcommand\pgph@installcite{%
441   \@for\pgph@conename:=\pgph@citecmds\do{%
442     \edef\pgph@cone{\expandafter\pgph@trimsp\expandafter{\pgph@conename}}}%
443     \expandafter\pgph@installciteone\expandafter{\pgph@cone}}%
444 }

```

The capture macro of a command is built on first use rather than declared: it is the same `\pgph@cap@citegen` call for all of them, and building it only for commands the document actually defines keeps the unused names of the list from costing anything. It must be `\gdef`, the installation happening inside the group of a proof.

```

445 \newcommand\pgph@installciteone[1]{%
446   \ifcsname #1\endcsname
447   \ifcsname pgph@cap@#1\endcsname\else
448     \expandafter\gdef\csname pgph@cap@#1\endcsname{\pgph@cap@citegen{#1}}%
449   \fi
450   \expandafter\ifx\csname #1\endcsname\expandafter\endcsname
451     \csname pgph@cap@#1\endcsname
452   \else
453     \expandafter\let\csname pgph@saved@#1\endcsname\expandafter\endcsname
454     \csname #1\endcsname
455     \expandafter\let\csname #1\endcsname\expandafter\endcsname
456     \csname pgph@cap@#1\endcsname
457   \fi
458 \fi
459 }

```

The biblatex recorder is registered at the end of the preamble, once biblatex has certainly been loaded (it is a preamble-only package) but while `\AtEveryCitekey` still accepts additions. It runs both at every citation and at every bibliography entry, so that a work only `\nocited`, or reached by `\usescite` alone, is still seen. It is not conditioned on the `cite` option, which governs the automatic capture of `\cite` only: `\usescite` and `\proofgraphciteedge` draw citation nodes without it, and those nodes want their tag and their anchor just the same.

```

460 \newif\ifpgph@blx
461 \AtEndPreamble{%
462   \ifpackageloaded{biblatex}{%
463     \pgph@blxtrue
464     \AtEveryCitekey{\pgph@blxrecord}%
465     \AtEveryBibitem{\pgph@blxrecord}%
466   }{}%
467 }

```

6.7 User commands

```

468 \newcommand\uses[1]{\pgph@recordref{#1}}

```

`\proofof` names the result a proof establishes. Inside a proof instance it records the label against the instance, so that the pin covers the whole proof, wherever in it the command stands, and is resolved (with every other label) when the graph is written. Outside one – in an `apxproof` deferred proof, whose owning result the

`\apxproofhook` already gives, or in the body of a result under `statements` – there is no instance to pin, and the source is redirected on the spot instead.

```

469 \newcommand\proofof[1]{%
470   \ifcsname pgph@powner@\pgph@curproof\endcsname
471     \expandafter\gdef\csname pgph@pofl@\pgph@curproof\endcsname{#1}%
472   \else
473     \ifcsname pgph@map@#1\endcsname
474       \xdef\pgph@curproof{\csname pgph@map@#1\endcsname}%
475     \fi
476   \fi
477 }
478 \newcommand\proofgraphedge[2]{%
479   \g@addto@macro\pgph@edges{\pgph@labeledge{#1}{#2}}%
480 }
481 \newcommand\pgph@labeledge[2]{%
482   \edef\pgph@ef{\pgph@resolve{#1}}%
483   \ifx\pgph@ef\@empty\else
484     \@for\pgph@one:=#2\do{%
485       \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
486       \pgph@plainedge{\pgph@ef}{\pgph@k}%
487     \fi
488 }

```

`\usescite`

`\proofgraphusesciteedge` The two manual counterparts of the automatic `\cite` capture, for a dependency on external work that no citation in a proof expresses: `\usescite`, the `\uses`
`\proofgraphciteedge` of citation nodes, records one from inside a proof, and `\proofgraphciteedge`,
`\pgph@citelabeledge` the `\proofgraphedge` of citation nodes, states one from anywhere. Both create

the citation node if the work has no node yet, take a comma-separated list of BibTeX keys and accept as an optional argument the note that would have been the optional argument of `\cite` (`[Thm~3]`), which labels the node and, as in the automatic capture, distinguishes one result of a work from another.

Being explicit, they do not depend on the `cite` option, which only says whether a plain `\cite` is captured on its own. `\usescite` shares the recording path of the captured citation commands, so a work reached both ways still yields a single node.

```

489 \NewDocumentCommand\usescite{0{}m}{\pgph@cap@citerec{#1}{#2}}
490 \NewDocumentCommand\proofgraphciteedge{0{}mm}{%
491   \g@addto@macro\pgph@edges{\pgph@citelabeledge{#1}{#2}{#3}}%
492 }

```

Resolution is deferred to emission, as for `\proofgraphedge`, so the result may be labelled anywhere in the document; the slot is therefore allocated then, which is harmless as slot numbers only have to be distinct. The edge is assembled by `\edef` rather than passed as macros, because `\pgph@citeedge` takes the `cite:<slot>` target apart with `xstring`, which reads its argument literally.

```

493 \newcommand\pgph@citelabeledge[3]{%
494   \edef\pgph@ef{\pgph@resolve{#2}}%
495   \ifx\pgph@ef\@empty\else
496     \@for\pgph@one:=#3\do{%
497       \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
498       \pgph@citeslot{\pgph@k}{#1}%
499       \edef\pgph@tmpce{%

```

```

500         \noexpand\pgph@citeedge{\pgph@ef}{cite:\pgph@curslot}}}%
501         \pgph@tmpce}%
502     \fi
503 }

```

`\proofgraphignore` The editing rules. Each undoes one of the recording commands and takes the arguments of the command it undoes: `\proofgraphignore` those of `\proofgraphedge`, `\proofgraphignorecite` the result first and what its proof uses second, and `\proofgraphignorecite` those of `\proofgraphciteedge`, the result and then the keys, with the note of the citation as an optional argument. So a rule is written by copying the dependency it is to remove, not by reading the arrow off the picture, which with the default `direction=usedby` runs the other way. The `direction` option is not consulted here at all: the filters work on the relation recorded, and `direction` settles only how it is drawn.

```

504 \newcommand\proofgraphignore[2]{%
505     \g@addto@macro\pgph@ignores{\pgph@ignorerule{#1}{#2}}}%
506 }
507 \NewDocumentCommand\proofgraphignorecite{0}{mm}{%
508     \g@addto@macro\pgph@ignores{\pgph@citeignorerule{#1}{#2}{#3}}}%
509 }
510 \newcommand\proofgraphexclude[1]{%
511     \g@addto@macro\pgph@excludes{\pgph@excluderule{#1}}}%
512 }

```

```

513 \newcommand\proofgraphstyle[2]{\global\@namedef{pgph@style@#1}{#2}}

```

External citation nodes (option `cite`) share one style, held in `\pgph@citestyle` and overridable with `\proofgraphstylecite`; it defaults to `shape=note`.

```

514 \newcommand\pgph@citestyle{shape=note}
515 \newcommand\proofgraphstylecite[1]{\renewcommand\pgph@citestyle{#1}}

```

6.8 Resolution helpers

These run at end of document, after the `.aux` has populated the map.

```

516 \newcommand\pgph@resolve[1]{%
517     \ifcsname pgph@map@#1\endcsname\csname pgph@map@#1\endcsname\fi
518 }
519 % The key under which a filter is consulted is that of the relation recorded,
520 % whose source is the \emph{result}; the two names below say which end is which,
521 % so that reading a rule is not a matter of remembering an order.
522 %     \begin{macrocode}
523 \newcommand\pgph@ignorerule[2]{%
524     \edef\pgph@ires{\pgph@resolve{#1}}\edef\pgph@idep{\pgph@resolve{#2}}}%
525     \ifx\pgph@ires\@empty\else\ifx\pgph@idep\@empty\else
526         \@namedef{pgph@ig@\pgph@ires @\pgph@idep}{}%
527     \fi\fi
528 }

```

A citation rule names the work depended on by key and note, the pair a citation node stands for, so it has to go through the slot allocator to learn which node that is. Asking for the slot of a work no proof cites allocates one, which costs nothing: slots become nodes only when an edge reaches them.

```

529 \newcommand\pgph@citeignorerule[3]{%

```

```

530 \edef\pgph@ires{\pgph@resolve{#2}}%
531 \ifx\pgph@ires\@empty\else
532   \@for\pgph@one:=#3\do{%
533     \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
534     \pgph@citeslot{\pgph@k}{#1}%
535     \@namedef{pgph@igc@pgph@ires @pgph@curslot}{}}%
536   \fi
537 }
538 \newcommand\pgph@excluderule[1]{%
539   \edef\pgph@x{\pgph@resolve{#1}}%
540   \ifx\pgph@x\@empty\else\@namedef{pgph@ex@\pgph@x}{}\fi
541 }

```

\pgph@checkrules An editing rule that matches nothing does nothing, and says nothing: the commonest way to get one is to write the two labels of `\proofgraphignore` in the order of the arrow drawn rather than of the dependency recorded, which leaves the graph exactly as it was, with no sign that a rule was even read. So the rules are replayed once the edges have been written, against the relations the document turned out to record, and each rule that met none is reported. When it is the swapped rule that would have matched, the report says so and gives the line to write, that being the mistake it almost always is.

The replay redefines the three rule macros and runs the same token lists again, so a rule cannot be checked in a way that differs from how it was applied, and adding a kind of rule cannot leave the check behind.

```

542 \newcommand\pgph@checkrules{%
543   \let\pgph@ignorerule\pgph@ignorecheck
544   \let\pgph@citeignorerule\pgph@citeignorecheck
545   \let\pgph@excluderule\pgph@excludecheck
546   \pgph@ignores
547   \pgph@excludes
548 }
549 \newcommand\pgph@nolabelwarn[2]{%
550   \PackageWarning{proofgraph}{%
551     No such result in the graph:\MessageBreak
552     \string#1\space names ‘#2’, which is not\MessageBreak
553     the label of any result drawn}%
554 }
555 \newcommand\pgph@ignorecheck[2]{%
556   \edef\pgph@ires{\pgph@resolve{#1}}\edef\pgph@idep{\pgph@resolve{#2}}%
557   \ifx\pgph@ires\@empty
558     \pgph@nolabelwarn\proofgraphignore{#1}%
559   \else\ifx\pgph@idep\@empty
560     \pgph@nolabelwarn\proofgraphignore{#2}%
561   \else\ifcsname pgph@rel@\pgph@ires @pgph@idep\endcsname\else
562     \ifcsname pgph@rel@\pgph@idep @pgph@ires\endcsname
563     \def\pgph@hint{%
564       it is ‘#2’ that depends on ‘#1’.\MessageBreak
565       The two labels go the way the dependency is\MessageBreak
566       recorded, the result first, and not the way\MessageBreak
567       the arrow is drawn. Write instead\MessageBreak
568       \string\proofgraphignore{#2}{#1}}%
569   \else
570     \def\pgph@hint{%

```

```

571      nothing records that ‘#1’ depends on ‘#2’}%
572    \fi
573    \PackageWarning{proofgraph}{%
574      No edge suppressed by\MessageBreak
575      \string\proofgraphignore{#1}{#2}:\MessageBreak
576      \pgph@hint}%
577  \fi\fi\fi
578 }
579 \newcommand\pgph@citeignorecheck[3]{%
580   \edef\pgph@ires{\pgph@resolve{#2}}%
581   \ifx\pgph@ires\@empty
582     \pgph@nolabelwarn\proofgraphignorecite{#2}%
583   \else
584     \edef\pgph@tmpnote{\detokenize{#1}}%
585     \ifx\pgph@tmpnote\@empty
586       \def\pgph@hint{}%
587     \else
588       \def\pgph@hint{\space with the note given}%
589     \fi
590     \@for\pgph@one:=#3\do{%
591       \edef\pgph@k{\expandafter\pgph@trimsp\expandafter{\pgph@one}}%
592       \pgph@citeslot{\pgph@k}{#1}%
593       \ifcsname pgph@relc@\pgph@ires @\pgph@curslot\endcsname\else
594         \PackageWarning{proofgraph}{%
595           No edge suppressed by\MessageBreak
596           \string\proofgraphignorecite{#2}{\pgph@k}:\MessageBreak
597           nothing records that ‘#2’ cites that work\pgph@hint}%
598       \fi}%
599   \fi
600 }
601 \newcommand\pgph@excludecheck[1]{%
602   \edef\pgph@x{\pgph@resolve{#1}}%
603   \ifx\pgph@x\@empty\pgph@nolabelwarn\proofgraphexclude{#1}\fi
604 }

```

6.9 Emission

`\pgph@dotes` Escape the two characters that are special in a Graphviz double-quoted string, so a theorem name, number or citation note containing them does not corrupt the `.dot`: a backslash is doubled and a double quote is backslash-escaped (in that order). `\pgph@bschar` and `\pgph@dqchar` hold the catcode-12 backslash and double quote to search for; `xstring`’s expansion mode is saved and restored around the two substitutions.

```

605 \begingroup
606   \catcode'\|=0 \catcode\'\"=12 \catcode'\|=12
607   \gdef\pgph@bschar{\}%
608   \gdef\pgph@dqchar{"}%
609 \endgroup
610 \newcommand\pgph@dotes[1]{%
611   \saveexpandmode\expandarg
612   \StrSubstitute{#1}{\pgph@bschar}{\pgph@bschar\pgph@bschar}[#1]%
613   \StrSubstitute{#1}{\pgph@dqchar}{\pgph@bschar\pgph@dqchar}[#1]%
614   \restoreexpandmode}

```

`\pgph@node` Emit one node line, honouring exclusions and per-environment styling. External (`cite:`) targets are emitted separately as a pass over edges. An *unnumbered* result (no number captured) is omitted: a results graph is about numbered statements, and unnumbered theorem-like environments are typically expository or repeated copies (e.g., a class-predefined `\spnewtheorem*{claim}`). `\proofgraphtrack{<env>}` overrides this, so a deliberately unnumbered environment is still drawn.

```

615 \newcommand\pgph@node[3]{%
616   \ifcsname pgph@ex@#1\endcsname\else
617     \edef\pgph@nm{\ifcsname pgph@num@#1\endcsname%
618       \csname pgph@num@#1\endcsname\fi}%
619     \ifx\pgph@nm\@empty
620       \ifcsname pgph@force@#2\endcsname
621         \pgph@emitnode{#1}{#2}{#3}%
622       \else
623         \@namedef{pgph@ex@#1}{}%
624       \fi
625     \else
626       \pgph@emitnode{#1}{#2}{#3}%
627     \fi
628   \fi
629 }
630 \newcommand\pgph@emitnode[3]{%
631   \edef\pgph@s{\ifcsname pgph@style@#2\endcsname
632     , \csname pgph@style@#2\endcsname\fi}%
633   \edef\pgph@dn{\ifcsname pgph@name@#1\endcsname
634     \csname pgph@name@#1\endcsname\else#3\fi}%
635   \edef\pgph@lbl{\pgph@dn\space\pgph@nm}%
636   \pgph@dotes\pgph@lbl
637   \immediate\write\pgph@out{%
638     \space\space"n#1" [label="\pgph@lbl"\pgph@s];}%
639   \ifcsname pgph@rmap@#1\endcsname
640     \immediate\write\pgph@mapout{%
641       \string\pgph@nodemap{n#1}{\csname pgph@rmap@#1\endcsname}}}%
642   \fi
643 }

```

`\pgph@edge` Emit one edge, resolving both ends and applying the self-loop, ignore and exclude `\pgph@setsrc` filters. Targets beginning with `cite:` denote external nodes.

The source is resolved by `\pgph@setsrc`. An edge captured in a proof has that proof instance as its source, and the result it belongs to is decided here: the label `\proofof` pinned the proof to, when there is one and it resolves, and otherwise the result the proof followed. An unresolvable pin falls back on that result rather than dropping the edge, so a mistyped or never-labelled `\proofof` loses the attribution but not the dependency. An edge recorded outside a proof (the body or the title of a result, an `apxproof` deferred proof) already carries a result id, which is its own source.

A sub-proof owns not a result but the proof it is nested in, so the owner is followed as a chain: at each step the pin of the instance reached wins if it resolves, and otherwise the walk goes on to its owner. The nearest pin therefore decides, an inner `\proofof` overriding an outer one for the references of the inner proof alone, and an outer one covering every nested reference no inner pin claims. The

walk always terminates: an owner is either an instance opened strictly earlier or a result id, which owns nothing.

```

644 \newcommand\pgph@setsrc[1]{%
645   \edef\pgph@esrc{#1}%
646   \pgph@srcwalk
647 }
648 \newcommand\pgph@srcwalk{%
649   \ifcsname pgph@owner@\pgph@esrc\endcsname
650     \let\pgph@srcnext\pgph@srcstep
651   \else
652     \let\pgph@srcnext\relax
653   \fi
654   \pgph@srcnext
655 }
656 \newcommand\pgph@srcstep{%
657   \let\pgph@epin\@empty
658   \ifcsname pgph@pofl@\pgph@esrc\endcsname
659     \edef\pgph@epin{csname pgph@pofl@\pgph@esrc\endcsname}%
660     \edef\pgph@epin{\pgph@resolve{\pgph@epin}}%
661   \fi
662   \ifx\pgph@epin\@empty
663     \edef\pgph@esrc{csname pgph@owner@\pgph@esrc\endcsname}%
664     \pgph@srcwalk
665   \else
666     \let\pgph@esrc\pgph@epin
667   \fi
668 }
669 \newcommand\pgph@edge[2]{%
670   \pgph@setsrc{#1}%
671   \ifx\pgph@esrc\@empty\else
672     \expandafter\pgph@edgeone\expandafter{\pgph@esrc}{#2}%
673   \fi
674 }
675 \newcommand\pgph@edgeone[2]{%
676   \IfBeginWith{#2}{cite:}%
677     {\pgph@citeedge{#1}{#2}}%
678     {\IfBeginWith{#2}{node:}%
679       {\StrBehind{#2}{node:}[\pgph@ntgt]%
680        \pgph@plainedgeid{#1}{\pgph@ntgt}}%
681       {\pgph@plainedge{#1}{#2}}}%
682 }

```

A target is normally a user label, resolved here; a `node:` one is a result id already, recorded by `\pgph@supportedge`, which knows the result it points at without its having to be labelled at all. Past that, the two are the same edge and take the same filters.

```

683 \newcommand\pgph@plainedge[2]{%
684   \edef\pgph@ptgt{\pgph@resolve{#2}}%
685   \ifx\pgph@ptgt\@empty\else
686     \pgph@plainedgeid{#1}{\pgph@ptgt}%
687   \fi
688 }
689 \newcommand\pgph@plainedgeid[2]{%
690   \edef\pgph@t{#2}%

```

```

691 \namedef{pgph@rel@#1@pgph@t}{}%
692 \ifcsname pgph@ex@#1\endcsname\else
693 \ifcsname pgph@ex@pgph@t\endcsname\else
694 \ifcsname pgph@ig@#1@pgph@t\endcsname\else
695 \ifbool{expr{ test {\ifnumequal{#1}{pgph@t}}
696 and test {\ifdefstring{pgph@selfloops}{remove}} } }%
697 {}{pgph@emittedge{#1}{npgph@t}}}%
698 \fi\fi\fi
699 }

```

`\pgph@emittedge` The relation recorded is always “the result whose proof we are in (the first argument) uses the target (the second)”. The `direction` option chooses the arrow orientation: `direction=usedby` (default) draws *target to result*, so an arrow `a->b` reads “a is used by b” (i.e. b relies on a); `direction=uses` draws *result to target* (“a uses b”). The filters above work on the recorded relation, so they are unaffected by the chosen orientation. `\pgph@writeedge` deduplicates.

```

700 \newcommand\pgph@emittedge[2]{%
701 \ifdefstring{pgph@direction}{uses}%
702 {pgph@writeedge{#1}{#2}}%
703 {pgph@writeedge{#2}{#1}}%
704 }
705 \newcommand\pgph@writeedge[2]{%
706 \ifcsname pgph@drawn@#1@#2\endcsname\else
707 \namedef{pgph@drawn@#1@#2}{}%
708 \immediate\writepgph@out{\space\space"#1" -> "#2";}%
709 \fi
710 }

```

`\pgph@citedge` A `cite:⟨slot⟩` target: recover the slot’s BibT_EX key, declare the external node once per slot, then draw the edge. The node *label* is built by `\pgph@setcitelabel`: it is the key by default, or, with `citelabel=tag`, the printed citation tag (the “[42]”/“[Knu74]” text), recovered either from a tag recorded by `\pgph@blxrecord` under biblatex or from the `\b@⟨key⟩` macro that `\bibcite` writes to the .aux (so two runs are needed, and it falls back to the key when no tag is known, e.g., on the first run or with citation packages neither path covers). The slot’s `\cite` note, when present, is added inside the tag. The node-to-label map records the *anchor* of the cited work, so all slots of one work hyperlink to the same bibliography entry.

```

711 \newcommand\pgph@citedge[2]{%
712 \StrBehind{#2}{cite:}[\pgph@slot]%
713 \namedef{pgph@relc@#1@pgph@slot}{}%
714 \ifcsname pgph@igc@#1@pgph@slot\endcsname\else
715 \ifcsname pgph@ex@#1\endcsname\else
716 \edef\pgph@key{\csname pgph@cskey@pgph@slot\endcsname}%
717 \ifcsname pgph@extnode@pgph@slot\endcsname\else
718 \namedef{pgph@extnode@pgph@slot}{}%
719 \expandafter\pgph@setcitelabel\expandafter{\pgph@slot}%
720 \pgph@dotesc\pgph@clabel
721 \immediate\writepgph@out{%
722 \space\space"c@pgph@slot"
723 [label="\pgph@clabel",pgph@citestyle];}%
724 \immediate\writepgph@mapout{%
725 \string\pgph@nodemapcite{c@pgph@slot}{pgph@citeanchor{pgph@key}}}%
726 \fi

```

```

727 \pgph@emittedge{n#1}{c@\pgph@slot}%
728 \fi\fi
729 }
730 \edef\pgph@obrace{\expandafter\@gobble\string\{ }
731 \newif\ifpgph@natnum
732 \newcommand\pgph@nil{ }

```

`\pgph@citesanitize` neutralizes the wrappers and spacing cruft that citation labels carry, so that an `\edef` of `\b@{key}` yields plain text: `hyperref's \hyper@link` (keep its printed last argument) and the `boxing/penalty/\ignorespaces` noise that `natbib` bakes into author lists.

```

733 \newcommand\pgph@citesanitize{%
734 \let\protect\@empty
735 \def\hyper@link[##1]##2##3##4{##4}%
736 \let\unskip\@empty \let\ignorespaces\@empty \let\unhbox\@empty
737 \let\penalty\@gobble \let\@M\@empty \let\voidb@x\@empty
738 \let\nobreakspace\space \let~\space
739 \def\hbox##1{##1}\def\mbox##1{##1}%
740 }

```

A `natbib \b@{key}` is `{(num)}{(year)}{(short)}{(long)}`. In numbers mode the tag is the number; in author–year mode it is the short author list and the year.

```

741 \def\pgph@natpick#1#2#3#4\pgph@nil{%
742 \ifpgph@natnum\def\pgph@x{#1}\else\def\pgph@x{#3 #2}\fi}
743 \newcommand\pgph@natdo{\expandafter\pgph@natpick\pgph@x}{ }\pgph@nil}

```

`\pgph@natnumset` Which of the two `natbib` modes is in force is read off `\ifNAT@numbers`, by comparing it with `\iftrue`. That comparison *must* live in a macro of its own rather than inline in `\pgph@setcitelabel`, because the code around it can be skipped by a false conditional: skipping does not expand anything, it merely counts conditional tokens, and the `\iftrue` here – an operand of `\ifx`, not a conditional – would be counted as one more `\if` to be closed. The skip would then eat one `\fi` too many and run past the end of `\pgph@setcitelabel` into its caller, silently swallowing the very writes that emit the citation node. Inside a macro the token is never seen by the skipping scanner.

```

744 \newcommand\pgph@natnumset{%
745 \pgph@natnumfalse
746 \@ifundefined{ifNAT@numbers}{}{%
747 \expandafter\ifx\csname ifNAT@numbers\endcsname\iftrue
748 \pgph@natnumtrue\fi}%
749 }

```

`\pgph@blxrecord` `biblatex` writes no `\b@{key}`: its tags are assembled while the citation is typeset, from data the `.bbl` holds. We therefore record them as they go by, from `\AtEveryCitekey` and `\AtEveryBibitem` (installed below), where the entry is the current one and its fields are readable. `labelnumber` carries the tag of the numeric styles and `labelalpha` that of the alphabetic ones; the author–year styles have neither, as their tag is a formatted name list rather than a field, and are left to the `key` fallback. A field that survives sanitizing with a backslash still in it is not plain text, and is likewise declined.

The same visit records the entry’s *anchor*, the name of the `hyperref` destination its bibliography entry carries, which `biblatex` does not spell `cite.<key>` as the `LATEX` kernel and `natbib` do but `cite.<refsection>@<key>`, the reference section being part

of the name because one key may have an entry in each. Naming the destination wrongly is not a visible failure but a misleading one: `hyperref` sends an unknown destination to the end of the document, so the node would link somewhere rather than nowhere. The anchor is recorded whatever `citelabel` says, the hyperlink being drawn either way.

```

750 \newcommand\pgph@blxrecord{%
751   \begingroup
752   \pgph@citesanitize
753   \edef\pgph@blxkey{\thefield{entrykey}}%
754   \pgph@gxdefcs{pgph@blxanchor@pgph@blxkey}%
755     {\the\c@refsection @pgph@blxkey}%
756   \ifdefstring{pgph@citelabel}{tag}{%
757     \edef\pgph@x{\thefield{labelnumber}}%
758     \ifx\pgph@x@empty \edef\pgph@x{\thefield{labelalpha}}\fi
759     \edef\pgph@dx{\detokenize\expandafter{pgph@x}}%
760     \ifx\pgph@x@empty\else
761       \IfSubStr\pgph@dx\@backslashchar{}{%
762         \pgph@gxdefcs{pgph@blxtag@pgph@blxkey}{pgph@x}%
763       }\fi
764     }{}%
765   \endgroup
766 }
```

The anchor of a key never met as a citation nor as a bibliography entry is not known; under `biblatex` we then assume the first reference section, which is the only one most documents have. A key absent from the bibliography has no destination under any name, so nothing is lost when the guess is wrong.

```

767 \newcommand\pgph@citeanchor[1]{%
768   \ifcsname pgph@blxanchor@#1\endcsname
769     \csname pgph@blxanchor@#1\endcsname
770   \else
771     \ifpgph@blx 0@ \fi#1%
772   \fi
773 }
```

A note is written the way a citation with a note is printed, inside the tag and after it: [Knu68, pp. 23–27], not the tag after the note. Its dashes are the two ligatures `LATEX` reads as dashes, `--` and `---`, which mean nothing to `Graphviz`; they are turned into the HTML character entities `Graphviz` decodes in an ordinary label, so that a page range set as [pp.~23--27] carries the same dash in the graph as in the document. The entities are built with `&` of category 12, as the escapes of `\pgph@dotesc` are, so that nothing in the label depends on the category codes in force where the graph is written.

```

774 \begingroup
775   \catcode'\&=12
776   \gdef\pgph@endash{&ndash;}%
777   \gdef\pgph@emdash{&mdash;}%
778 \endgroup
779 \newcommand\pgph@dashes[1]{%
780   \saveexpandmode\expandarg
781   \StrSubstitute{#1}{---}{pgph@emdash}[#1]%
782   \StrSubstitute{#1}{--}{pgph@endash}[#1]%
783   \restoreexpandmode}
```

```

784 \newif\ifpgph@ctag
785 \newcommand\pgph@setcitelabel[1]{%
786   \edef\pgph@ckey{\csname pgph@cskey@#1\endcsname}%
787   \edef\pgph@cbase{\pgph@ckey}%
788   \pgph@ctagfalse
789   \ifdefstring{\pgph@citelabel}{tag}{%
790     \ifcsname pgph@blxtag@\pgph@ckey\endcsname
791       \edef\pgph@cbase{\csname pgph@blxtag@\pgph@ckey\endcsname}%
792       \pgph@ctagtrue
793     \else
794       \ifcsname b@\pgph@ckey\endcsname
795         \global\let\pgph@cbase\relax
796         \begingroup
797           \pgph@citesanitize
798           \edef\pgph@x{\csname b@\pgph@ckey\endcsname}%
799           \edef\pgph@dx{\detokenize\expandafter{\pgph@x}}%
800           \IfBeginWith\pgph@dx\pgph@obrace{%
801             \pgph@natnumset
802             \pgph@natdo
803             \edef\pgph@dx{\detokenize\expandafter{\pgph@x}}%
804           }{%
805             \ifx\pgph@x\@empty\else
806               \IfSubStr\pgph@dx\@backslashchar}{%
807                 \global\let\pgph@cbase\pgph@x}%
808             \fi
809           \endgroup
810           \ifx\pgph@cbase\relax\else
811             \let\pgph@cbase\pgph@cbase
812             \pgph@ctagtrue
813           \fi
814         \fi\fi
815       }{%
816         \edef\pgph@cnote{%
817           \ifcsname pgph@citenote@#1\endcsname
818             , \csname pgph@citenote@#1\endcsname\fi}%
819         \ifx\pgph@cnote\@empty\else\pgph@dashedc\pgph@cnote\fi
820         \edef\pgph@clabel{%
821           \ifpgph@ctag[\fi\pgph@cbase\pgph@cnote\ifpgph@ctag]\fi}%
822 }

```

`\pgph@render` `autorun` must not re-render a graph that has not changed. Rendering unconditionally is not merely wasteful: the image `Graphviz` produces is not guaranteed to be reproducible (its `cairo`-based outputs, `pdf` among them, carry a creation date), so an unconditional run makes the embedded image differ at every compilation. A build tool such as `latexmk` then sees an input file that never settles and reruns until it gives up, reporting that too many passes were needed.

We therefore render only when something relevant has changed. The signature of a rendering is the checksum of the `.dot` file just written, together with the engine and the format, which the `.dot` does not record; it is stored in the `.aux` by `\pgph@lastrender` and compared with the signature of the next run. `Graphviz` is run when the signature differs, when no signature is known yet (first run), when either output file is missing, and, as a safe fallback, when the engine provides no file checksum.

Both signatures are `\detokenized` before being compared. The checksum comes out of `\pdf@filemdfivesum` as characters of category *other*, while the same string read back from the `.aux` has its letters as category *letter*, so an `\ifx` on the raw strings would never see two runs as equal.

```

823 \let\pgph@prevsig\relax
824 \newcommand\pgph@lastrender[1]{\xdef\pgph@prevsig{\detokenize{#1}}}
825 \newcommand\pgph@render{%
826   \immediate\write18{\pgph@engine\space -T\pgph@format\space
827     \pgph@file.dot -o \pgph@file-proofgraph.\pgph@format}%
828   \immediate\write18{\pgph@engine\space -Tplain\space
829     \pgph@file.dot -o \pgph@file-proofgraph.plain}%
830 }
831 \newcommand\pgph@maybrender{%
832   \edef\pgph@sig{\ifdefined\pdf@filemdfivesum
833     \pdf@filemdfivesum{\pgph@file.dot}\fi}%
834   \ifx\pgph@sig\@empty
835     \pgph@render
836   \else
837     \edef\pgph@sig{\pgph@sig-\pgph@engine-\pgph@format}%
838     \edef\pgph@sig{\expandafter\detokenize\expandafter{\pgph@sig}}%
839     \IfFileExists{\pgph@file-proofgraph.\pgph@format}%
840       {\IfFileExists{\pgph@file-proofgraph.plain}%
841         {\ifx\pgph@sig\pgph@prevsig\else\pgph@render\fi}%
842         {\pgph@render}}%
843       {\pgph@render}%

```

The signature is recorded whether or not we have just rendered: either way the output files on disk are the ones this `.dot` produces. The write must be `\immediate`, as `\pgph@emit` runs at end of document, where a deferred write would have no shipout left to carry it.

```

844   \if@filesw
845     \immediate\write\@auxout{\string\pgph@lastrender{\pgph@sig}}%
846   \fi
847 \fi
848 }

```

`\pgph@emit` Open the file, resolve editing rules, write nodes then edges, close, and optionally run Graphviz.

```

849 \newwrite\pgph@out
850 \newwrite\pgph@mapout
851 \newcommand\pgph@emit{%
852   \immediate\openout\pgph@out=\pgph@file.dot\relax
853   \immediate\openout\pgph@mapout=\pgph@file-proofgraph.map\relax
854   \immediate\write\pgph@out{digraph proofgraph \pgph@ob}%
855   \immediate\write\pgph@out{\space\space rankdir="\pgph@rankdir";}%
856   \begingroup
857     \pgph@ignores
858     \pgph@excludes
859     \pgph@nodes
860     \pgph@edges
861     \pgph@checkrules
862   \endgroup
863   \immediate\write\pgph@out{\pgph@cb}%
864   \immediate\closeout\pgph@out

```

```

865 \immediate\closeout\pgph@mapout
866 \ifpgph@autorun\pgph@maybrender\fi
867 }

```

The emission is registered at `\begin{document}` rather than at load time so that it is added to the end-document hook *after* packages loaded later (notably `apxproof`, whose appendix, with the deferred proofs and their references, is built at end of document). This way the graph is written only once that material has been typeset, and the emission does not disturb the verbatim replay of deferred proofs.

```

868 \AtBeginDocument{\AtEndDocument{\pgph@emit}}

```

If TikZ is available, load its `calc` library now (at `\begin{document}`, where category codes are normal) so that the hyperlink overlay in `\proofgraph` can use it without reading a file mid-document.

```

869 \AtBeginDocument{\@ifpackageloaded{tikz}{\usetikzlibrary{calc}}{}}

```

`\pgphnodemap` One entry of the node-to-target map written by `\pgph@emit`: it records, for a `\pgphnodemapcite` Graphviz node identifier, what the node should link to. `\pgphnodemap` records the user *label* of the result a node represents; `\pgphnodemapcite` records the *anchor* of an external citation node, so that (when the cited work is in the same document) the node can link to its bibliography entry, `hyperref`'s `cite.<anchor>` destination, the anchor being the BIB_T_EX key except under `biblatex` (see `\pgph@citeanchor`). The map is read back when the graph is embedded, to turn each node into a hyperlink.

```

870 \newcommand\pgphnodemap[2]{\global\@namedef{pgph@lbl@#1}{#2}}
871 \newcommand\pgphnodemapcite[2]{\global\@namedef{pgph@cbl@#1}{#2}}

```

`\proofgraph` Include the rendered graph (from a previous run) in `autorun` mode. When `hyperref` and `TikZ` are both available (and `hyperlinks` is not switched off), each node is made into an internal hyperlink to what it represents: a result node links to the result (its `\label`), and an external citation node (from the `cite` option) links to the cited work's bibliography entry, the `cite.<anchor>` destination `hyperref` gave it, when that work is cited in the same document. `Graphviz` produces a `.pdf` whose link annotations would be lost by `\includegraphics` (they sit inside a form `XOBJECT`), so instead we overlay transparent link areas at the node positions, which we read from the `-Tplain` output. Without those packages we fall back to a plain inclusion.

```

872 \newif\ifpgph@canlink
873 \newif\ifpgph@link
874 \newsavebox\pgph@imgbox
875 \newsavebox\pgph@natbox
876 \newread\pgph@plainin
877 \def\pgph@stop{}
878 \newcommand\proofgraph[1][{}]{%
879   \@ifundefined{includegraphics}{%
880     {\PackageError{proofgraph}{\string\proofgraph space requires the
881       graphicx package}{Add \string\usepackage{graphicx} to your preamble
882       to embed the rendered graph.}}}%
883     {\IfFileExists{\pgph@file-proofgraph.\pgph@format}{%
884       {\pgph@includegraph{#1}}}%
885       {\PackageWarning{proofgraph}{Rendered graph
886         \pgph@file-proofgraph.\pgph@format space not found; compile with
887         shell-escape and the autorun option, then re-run}}}%

```

```

888 }
889 \newcommand\pgph@includegraph[1]{%
890   \pgph@canlinkfalse
891   \ifpgph@hyperlinks
892     \ifpackageloaded{hyperref}%
893       {\ifpackageloaded{tikz}%
894         {\IfFileExists{\pgph@file-proofgraph.plain}%
895          {\pgph@canlinktrue}{}}{}}}%
896   \fi
897   \ifpgph@canlink
898     \pgph@graphlinked{#1}%
899   \else
900     \includegraphics[#1]{\pgph@file-proofgraph.\pgph@format}%
901   \fi
902 }

```

The linked version. We read the node-to-target map under safe category codes (a label may contain `_` or, under `babel`, an active `:`), measure the rendered image, place it in a TikZ node, then read the `-Tplain` file and drop one transparent link box on each mapped node. Graphviz draws the graph inset by a margin, so the image is larger than the layout bounding box the `-Tplain` coordinates live in; we recover that margin by also measuring the image at its *natural* size and comparing it with the `-Tplain` graph size (the `graph` line gives it in inches, hence the factor 72). Node positions are then expressed as fractions of the natural image, so they line up with the drawn nodes whatever size the image is finally scaled to.

```

903 \newcommand\pgph@graphlinked[1]{%
904   \begingroup
905     \catcode'\:=12 \catcode'\_:=12 \catcode'\;=12 \catcode'\!=12
906     \catcode'\?=12 \catcode'\&=12 \catcode'\^=12 \catcode'\~=12
907     \InputIfFileExists{\pgph@file-proofgraph.map}{}{}%
908   \endgroup
909   \sbox\pgph@imgbox{%
910     \includegraphics[#1]{\pgph@file-proofgraph.\pgph@format}}%
911   \sbox\pgph@natbox{%
912     \includegraphics{\pgph@file-proofgraph.\pgph@format}}%
913   \edef\pgph@imgw{\the\wd\pgph@imgbox}%
914   \edef\pgph@imgh{\the\dimexpr\ht\pgph@imgbox+\dp\pgph@imgbox\relax}%
915   \edef\pgph@natw{\the\wd\pgph@natbox}%
916   \edef\pgph@nath{\the\dimexpr\ht\pgph@natbox+\dp\pgph@natbox\relax}%
917   \begin{tikzpicture}
918     \node[inner sep=\z@,outer sep=\z@](pgph@P){\usebox\pgph@imgbox};
919     \openin\pgph@plainin=\pgph@file-proofgraph.plain\relax
920     \pgph@loopplain
921     \closein\pgph@plainin
922   \end{tikzpicture}%
923 }
924 \newcommand\pgph@loopplain{%
925   \unless\ifeof\pgph@plainin
926     \read\pgph@plainin to \pgph@line
927     \pgph@procline
928     \expandafter\pgph@loopplain
929   \fi
930 }
931 \newcommand\pgph@procline{%

```

```

932 \IfBeginWith{\pgph@line}{graph }%
933   {\expandafter\pgph@dograph\pgph@line\pgph@stop}%
934   {\IfBeginWith{\pgph@line}{node }%
935     {\expandafter\pgph@donode\pgph@line\pgph@stop}{}}%
936 }
937 \def\pgph@dograph graph #1 #2 #3 #4\pgph@stop{%
938   \def\pgph@gw{#2}\def\pgph@gh{#3}}
939 \def\pgph@donode node #1 #2 #3 #4 #5 #6\pgph@stop{%
940   \StrDel{#1}{"}[\pgph@nid]%
941   \pgph@linkfalse
942   \ifcsname pgph@lbl@\pgph@nid\endcsname
943     \edef\pgph@tgt{\csname pgph@lbl@\pgph@nid\endcsname}%
944     \edef\pgph@thislink{\noexpand\hyperref[\pgph@tgt]}%
945     \pgph@linktrue
946   \else
947     \ifcsname pgph@clbl@\pgph@nid\endcsname
948       \edef\pgph@tgt{\csname pgph@clbl@\pgph@nid\endcsname}%
949       \edef\pgph@thislink{\noexpand\hyperlink{cite.\pgph@tgt}}%
950       \pgph@linktrue
951     \fi
952   \fi
953   \ifpgph@link
954     % Per-side Graphviz margin: half the natural-minus-layout size.
955     \pgfmathsetmacro\pgph@mx{(\pgph@natw-\pgph@gw*72)/2}%
956     \pgfmathsetmacro\pgph@my{(\pgph@nath-\pgph@gh*72)/2}%
957     % Node centre as a fraction of the natural image.
958     \pgfmathsetmacro\pgph@fx{(\pgph@mx+#2*72)/\pgph@natw}%
959     \pgfmathsetmacro\pgph@fy{(\pgph@my+#3*72)/\pgph@nath}%
960     \pgfmathsetlengthmacro\pgph@bw{#4*72/\pgph@natw*\pgph@imgw*0.92}%
961     \pgfmathsetlengthmacro\pgph@bh{#5*72/\pgph@nath*\pgph@imgh*0.9}%
962     \node[anchor=center]
963       at ($($\pgph@P.south west)! \pgph@fx!(\pgph@P.south east)$)%
964       !\pgph@fy!($(\pgph@P.north west)! \pgph@fx!(\pgph@P.north east)$)$)
965       {\pgph@thislink{\phantom{\rule{\pgph@bw}{\pgph@bh}}}};%
966   \fi
967 }
968 \end{package}

```

Index

Numbers written in *italic* refer to the page where the corresponding entry is described; numbers underlined refer to the code line of the definition; numbers in roman refer to the code lines where the entry is used.

Symbols		B	
\!	905	\begin	522, 917
\"	606	C	
\&	775, 906	\c@refsection	755
\:	905	\catcode	34, 35, 606, 775, 905, 906
\;	905	\closein	921
\?	906	\Cref	339
\@M	737	\cref	338
\@Opargbegintheorem	328, 329, 330	\cs	17
\@auxout	57, 845	\csundef	93, 101
\@backslashchar	761, 806	D	
\@begintheorem	152, 153, 155, 159	\DeclareBoolOption	21, 26, 28, 29
\@currentlabel	201, 215	\DeclareStringOption	20, 22, 23, 24, 25, 27, 30
\@ifpackageloaded	5, 376, 462, 869, 892, 893	\detokenize	347, 418, 584, 759, 799, 803, 824, 838
\@ifundefined	6, 10, 345, 746, 879	\dimexpr	914, 916
\@namedef	48, 50, 64, 94, 102, 108, 198, 513, 526, 535, 540, 623, 691, 707, 713, 718, 870, 871	\dp	914, 916
\@opargbegintheorem	154, 163, 164, 165, 317, 318, 319	E	
\@outputpage	238, 239	\emph	520
\@protected@testopt	347	\end	922
\@spbegintheorem	169, 170, 171	\endproof	7, 8, 11
\@spopargbegintheorem	174, 175, 176, 322, 323, 324	\expandarg	611, 780
\[	34	\ExplSyntaxOff	18
\]	606	\ExplSyntaxOn	16
\{	35, 730	H	
\}	35, 607	\hbox	186, 341, 739
\]	34	\ht	914, 916
\^	906	\hyper@@link	735
_	905	\hyperlink	949
\	606	\hyperref	944
\~	906	I	
A		\if@filesw	844
\AddToHook	247	\IfBeginWith	62, 676, 678, 800, 932, 934
\apxproofhook	380	\IfBooleanTF	263
\AtBeginDocument	128, 244, 375, 868, 869	\ifboolexpr	371, 695
\AtBeginEnvironment	109, 350	\ifdefstring	371, 696, 701, 756, 789
\AtEndDocument	868	\ifeof	925
\AtEndPreamble	461	\IfFileExists	839, 840, 883, 894
\AtEveryBibitem	465	\IfHookExistsTF	245, 246
\AtEveryCitekey	464	\ifnumequal	695
\autoref	339	\ifpgph@apxproof	367
\axp@forward@link	194, 214	\ifpgph@autorun	866

<code>\ifpgph@blx</code>	460, 771	<code>\pgph@beginresult</code>	109, 130
<code>\ifpgph@canlink</code>	872, 897	<code>\pgph@beginstatement</code>	149, 311
<code>\ifpgph@cite</code>	284	<code>\pgph@bh</code>	961, 965
<code>\ifpgph@ctag</code>	784, 821	<code>\pgph@blxkey</code>	753, 754, 755, 762
<code>\ifpgph@hyperlinks</code>	891	<code>\pgph@blxrecord</code>	464, 465, 750
<code>\ifpgph@link</code>	873, 953	<code>\pgph@blxtrue</code>	463
<code>\ifpgph@natnum</code>	731, 742	<code>\pgph@bschar</code>	612, 613
<code>\ifpgph@nocapture</code>	229, 236, 405	<code>\pgph@bw</code>	960, 965
<code>\ifpgph@statements</code>	149	<code>\pgph@candidates</code> 59, 65, 66, 68, 123, 124	
<code>\IfSubStr</code>	348, 761, 806	<code>\pgph@canlinkfalse</code>	890
<code>\iftrue</code>	747	<code>\pgph@canlinktrue</code>	895
<code>\ignorespaces</code>	736	<code>\pgph@cap@autoref</code>	256
<code>\includegraphics</code>	900, 910, 912	<code>\pgph@cap@cite@noopt</code>	396, 403
<code>\InputIfFileExists</code>	907	<code>\pgph@cap@cite@opt</code>	396, 397
J		<code>\pgph@cap@cite@opti</code>	398, 399
<code>\jobname</code>	32	<code>\pgph@cap@cite@optii</code>	398, 401
L		<code>\pgph@cap@cite@x</code>	392, 393
<code>\label</code>	147, 148	<code>\pgph@cap@citegen</code>	390, 448
M		<code>\pgph@cap@citerec</code>	
<code>\mbox</code>	739		400, 402, 403, 404, 489
<code>\meaning</code>	346	<code>\pgph@cap@Cref</code>	255
<code>\MessageBreak</code>	551, 552, 564, 565, 566, 567, 574, 575, 595, 596	<code>\pgph@cap@ceref</code>	254
N		<code>\pgph@cap@eqref</code>	258
<code>\newcount</code>	39, 40, 41	<code>\pgph@cap@ref</code>	253
<code>\NewDocumentCommand</code>		<code>\pgph@cap@vref</code>	260
.....	261, 334, 489, 490, 507	<code>\pgph@cap@zceref</code>	261
<code>\newif</code> 236, 367, 460, 731, 784, 872, 873		<code>\pgph@cb</code>	37, 863
<code>\newread</code>	876	<code>\pgph@cbase</code>	787, 791, 811, 821
<code>\newsavebox</code>	874, 875	<code>\pgph@cbaseg</code>	795, 807, 810, 811
<code>\newtheorem</code>	73, 74	<code>\pgph@checkrules</code>	542, 861
<code>\nobreakspace</code>	738	<code>\pgph@citeanchor</code>	725, 767
<code>\node</code>	918, 962	<code>\pgph@citecmd</code>	391, 395
<code>\noexpand</code> . 143, 224, 395, 500, 944, 949		<code>\pgph@citecmds</code>	431
O		<code>\pgph@citeedge</code>	500, 677, 711
<code>\openin</code>	919	<code>\pgph@citeignorecheck</code>	544, 579
P		<code>\pgph@citeignorerule</code> ..	508, 529, 544
<code>\PackageError</code>	880	<code>\pgph@citelabel</code>	756, 789
<code>\PackageWarning</code> ...	550, 573, 594, 885	<code>\pgph@citelabeledge</code>	489
<code>\pdf@filemdfivesum</code>	832, 833	<code>\pgph@citesanitize</code> . 415, 733, 752, 797	
<code>\penalty</code>	737	<code>\pgph@citeslot</code> 409, 413, 498, 534, 592	
<code>\pgfmathsetlengthmacro</code>	960, 961	<code>\pgph@citeslotcnt</code>	41, 422, 423
<code>\pgfmathsetmacro</code> ..	955, 956, 958, 959	<code>\pgph@citestyle</code>	514, 515, 723
<code>\pgph@adddge</code>	132, 223, 233, 410	<code>\pgph@ckey</code> 786, 787, 790, 791, 794, 798	
<code>\pgph@apxinit</code>	376, 379	<code>\pgph@clabel</code>	720, 723, 820
<code>\pgph@apxproofcapture</code>	379	<code>\pgph@class@endproof</code>	7, 11
<code>\pgph@apxprooftrue</code>	376	<code>\pgph@class@proof</code>	7, 11
<code>\pgph@auxmap</code>	48	<code>\pgph@cnote</code>	816, 819, 821
<code>\pgph@beginproof</code>	253, 350, 356	<code>\pgph@cone</code>	442, 443
		<code>\pgph@conename</code>	441, 442
		<code>\pgph@ctagfalse</code>	788
		<code>\pgph@ctagtrue</code>	792, 812
		<code>\pgph@curproof</code>	
		..	47, 139, 184, 230, 233, 298,

299, 302, 306, 312, 313, 382, 383, 384, 406, 410, 470, 471, 474	\pgph@idep
\pgph@curresult	. 524, 525, 526, 556, 559, 561, 562
. 46, 132, 141, 143, 182, 184, 191, 196, 198, 202, 209, 210, 211, 216, 287, 296, 304, 312	\pgph@ignorecheck 543, 555
\pgph@curslot 410, 420, 423, 424, 425, 427, 500, 535, 593	\pgph@ignorerule 505, 523, 543
\pgph@dashesc 779, 819	\pgph@ignores 44, 505, 508, 546, 857
\pgph@defaultenvs 86, 129	\pgph@imgbox 874, 909, 913, 914, 918
\pgph@defaulttexclude 89, 105	\pgph@imggh 914, 961
\pgph@direction 701	\pgph@imgw 913, 960
\pgph@dn 633, 635	\pgph@includefgraph 884, 889
\pgph@dograph 933, 937	\pgph@install 265, 278, 279, 280
\pgph@dohook 106, 114, 116	\pgph@installcapture
\pgph@donode 935, 939	 185, 282, 307, 314, 385
\pgph@dotesc 605, 636, 720	\pgph@installcite 284, 431
\pgph@dqchar 613	\pgph@installciteone 443, 445
\pgph@dx 759, 761, 799, 800, 803, 806	\pgph@installrefs 277, 283
\pgph@edge 224, 644	\pgph@ires
\pgph@edgeone 672, 675	. 524, 525, 526, 530, 531, 535, 556, 557, 561, 562, 580, 581, 593
\pgph@edges 43, 225, 479, 491, 860	\pgph@k 232, 233, 408, 409, 485, 486, 497, 498, 533, 534, 591, 592, 596
\pgph@ef 482, 483, 486, 494, 495, 500	\pgph@key 716, 725
\pgph@emdash 777, 781	\pgph@labeledge 479, 481
\pgph@emit 849	\pgph@lastrender 823
\pgph@emitedge 697, 700, 727	\pgph@lbl 635, 636, 638
\pgph@emitnode 621, 626, 630	\pgph@line 926, 932, 933, 934, 935
\pgph@encproof 131, 132, 139	\pgph@linkfalse 941
\pgph@endash 776, 782	\pgph@linktrue 945, 950
\pgph@engine 826, 828, 837	\pgph@loopplain 920, 924, 928
\pgph@epin 657, 659, 660, 662, 666	\pgph@maplabel 147, 148, 208
\pgph@esrc 645, 649, 658, 659, 663, 666, 671, 672	\pgph@mapout 640, 724, 850, 853, 865
\pgph@excludecheck 545, 601	\pgph@maybehook 112, 125
\pgph@excluderule 511, 538, 545	\pgph@maybrender 823, 866
\pgph@excludes 45, 511, 547, 858	\pgph@mx 955, 958
\pgph@file 32, 827, 829, 833, 839, 840, 852, 853, 883, 886, 894, 900, 907, 910, 912, 919	\pgph@my 956, 959
\pgph@format 826, 827, 837, 839, 883, 886, 900, 910, 912	\pgph@natbox 875, 911, 915, 916
\pgph@fx 958, 963, 964	\pgph@natdo 743, 802
\pgph@fy 959, 964	\pgph@nath 916, 956, 959, 961
\pgph@gh 938, 956	\pgph@natnumfalse 745
\pgph@gnotestr 416, 418, 426, 427	\pgph@natnumset 744, 801
\pgph@graphlinked 898, 903	\pgph@natnumtrue 748
\pgph@gw 938, 955	\pgph@natpick 741, 743
\pgph@gxdefcs	\pgph@natw 915, 955, 958, 960
. 388, 424, 425, 427, 754, 762	\pgph@ncur 296, 301
\pgph@hint 563, 570, 576, 586, 588, 597	\pgph@newproof 294, 295, 302, 304, 306
\pgph@hookcite 388	\pgph@nid 940, 942, 943, 947, 948
\pgph@hooknames 130	\pgph@nil 732, 741, 743
\pgph@hookproofhead 308, 316	\pgph@nm 617, 619, 635
\pgph@idcnt 39, 140, 141	\pgph@nocapturefalse 242
	\pgph@nocapturetrue 240, 247
	\pgph@node 143, 615
	\pgph@nodes 42, 136, 859
	\pgph@nolabelwarn
	 549, 558, 560, 582, 603
	\pgph@nomatch 300, 310

<code>\pgph@npres</code>	297, 301	<code>\pgph@savetlabel</code>	147, 221
<code>\pgph@nt@plain</code>	74, 75	<code>\pgph@scanproofarg</code>	320, 325, 331, <u>334</u>
<code>\pgph@nt@star</code>	74, 77	<code>\pgph@scanrefs</code>	156, 166, 177, <u>181</u>
<code>\pgph@ntgt</code>	679, 680	<code>\pgph@selfloops</code>	696
<code>\pgph@ob</code>	36, 854	<code>\pgph@setcitelabel</code>	<u>711</u>
<code>\pgph@obrace</code>	730, 800	<code>\pgph@setmap</code>	51, 209
<code>\pgph@one</code>	91, 92, 93, 94, 95, 99, 100, 101, 102, 120, 121, 122, 124, 125, 231, 232, 369, 370, 407, 408, 484, 485, 496, 497, 532, 533, 590, 591	<code>\pgph@setpowner</code>	289, 302, 304
<code>\pgph@orig@bt</code>	153, 157, 160	<code>\pgph@setpres</code>	286, 295, 313, 384
<code>\pgph@orig@newtheorem</code>	73, 76, 77	<code>\pgph@setsrc</code>	<u>644</u>
<code>\pgph@orig@obt</code>	164, 167	<code>\pgph@sig</code>	418, 419, 420, 424, 832, 834, 837, 838, 841, 845
<code>\pgph@orig@obtp</code>	329, 331	<code>\pgph@slot</code>	712, 713, 714, 716, 717, 718, 719, 722, 725, 727
<code>\pgph@orig@obtp</code>	318, 320	<code>\pgph@spnt@plain</code>	81, 82
<code>\pgph@orig@outputpage</code>	238, 241	<code>\pgph@spnt@star</code>	81, 84
<code>\pgph@orig@spbt</code>	170, 172	<code>\pgph@srcnext</code>	650, 652, 654
<code>\pgph@orig@spnewtheorem</code>	79, 83, 84	<code>\pgph@srcstep</code>	650, 656
<code>\pgph@orig@spobt</code>	175, 178	<code>\pgph@srcwalk</code>	646, 648, 664
<code>\pgph@orig@spobtp</code>	323, 326	<code>\pgph@stop</code>	877, 933, 935, 937, 939
<code>\pgph@out</code>	637, 708, 721, 849, 852, 854, 855, 863, 864	<code>\pgph@supportedge</code>	130, 145
<code>\pgph@pe</code>	370, 371, 372	<code>\pgph@t</code>	690, 691, 693, 694, 695, 697
<code>\pgph@plainedge</code>	486, 681, 683	<code>\pgph@testoptstr</code>	347, 348
<code>\pgph@plainedgeid</code>	680, 686, 689	<code>\pgph@tgt</code>	943, 944, 948, 949
<code>\pgph@plainin</code>	876, 919, 921, 925, 926	<code>\pgph@thislink</code>	944, 949, 965
<code>\pgph@prependnode</code>	135, 144	<code>\pgph@thisname</code>	61, 62, 63, 64, 66, 68
<code>\pgph@prevsig</code>	823, 824, 841	<code>\pgph@tmp</code>	142, 144
<code>\pgph@procline</code>	927, 931	<code>\pgph@tmpce</code>	499, 501
<code>\pgph@proofcnt</code>	40, 293, 294	<code>\pgph@tmpedge</code>	224, 226
<code>\pgph@proofenvs</code>	362, 369	<code>\pgph@tmphn</code>	199, 200
<code>\pgph@proofmeaning</code>	346, 348	<code>\pgph@tmpid</code>	52, 53, 54, 55, 57
<code>\pgph@proofopt</code>	357, 360	<code>\pgph@tmpname</code>	195, 197
<code>\pgph@ptgt</code>	684, 685, 686	<code>\pgph@tmpnote</code>	584, 585
<code>\pgph@rankdir</code>	855	<code>\pgph@tmpnum</code>	201, 203, 215, 217
<code>\pgph@recordref</code>	228, 253, 254, 255, 257, 259, 260, 262, 468	<code>\pgph@trimsp</code>	16, 92, 100, 232, 370, 408, 442, 485, 497, 533, 591
<code>\pgph@recordresult</code>	130	<code>\pgph@wrapoutputpage</code>	237, 248, 250
<code>\pgph@redefproof</code>	349, 353	<code>\pgph@wrapproof</code>	<u>334</u>
<code>\pgph@register</code>	59, 76, 83, 95, 122	<code>\pgph@wrapproofone</code>	344, 372
<code>\pgph@render</code>	823	<code>\pgph@wrapproofs</code>	368, 377
<code>\pgph@replay</code>	394, 400, 402, 403	<code>\pgph@writeedge</code>	<u>700</u>
<code>\pgph@resolve</code>	382, 482, 494, 516, 524, 530, 539, 556, 580, 602, 660, 684	<code>\pgph@x</code>	539, 540, 602, 603, 742, 743, 757, 758, 759, 760, 762, 798, 799, 803, 805, 807
<code>\pgph@resolveregistration</code>	119, 129	<code>\pgph@zcpproofof</code>	334, 340
<code>\pgph@s</code>	631, 638	<code>\pgph@nodemap</code>	641, <u>870</u>
<code>\pgph@savet@autoref</code>	257	<code>\pgph@nodemapcite</code>	725, <u>870</u>
<code>\pgph@savet@Cref</code>	255	<code>\phantom</code>	965
<code>\pgph@savet@cref</code>	254	<code>\ProcessKeyvalOptions</code>	31
<code>\pgph@savet@eqref</code>	259	<code>\proof</code>	7, 8, 11
<code>\pgph@savet@ref</code>	253	<code>\proofgraph</code>	8, <u>872</u>
<code>\pgph@savet@vref</code>	260	<code>\proofgraphcitecommand</code>	8, <u>431</u>
<code>\pgph@savet@zcreef</code>	263	<code>\proofgraphciteedge</code>	6, 28, <u>489</u>
		<code>\proofgraphedge</code>	6, 478
		<code>\proofgraphexclude</code>	8, <u>504</u> , 603

<code>\proofgraphignore</code>	7, 504, 558, 560, 568, 575
<code>\proofgraphignorecite</code>	8, 504, 582, 596
<code>\proofgraphprovenv</code>	7, 24, 362
<code>\proofgraphstyle</code>	8, 513
<code>\proofgraphstylecite</code>	8, 515
<code>\proofgraphtrack</code>	7, 14, 86
<code>\proofgraphuntrack</code>	7, 14, 86, 365
<code>\proofof</code>	6, 334, 338, 339, 469
<code>\protect</code>	193, 213, 337, 734
<code>\protected</code>	253, 254, 255, 256, 258, 260
<code>\protected@edef</code>	142, 224
<code>\protected@write</code>	57
R	
<code>\read</code>	926
<code>\ref</code>	338
<code>\restoreexpandmode</code>	614, 783
<code>\rule</code>	965
S	
<code>\saveexpandmode</code>	611, 780
<code>\sbox</code>	909, 911
<code>\setbox</code>	186, 341
<code>\SetupKeyvalOptions</code>	19
<code>\spnewtheorem</code>	78, 79, 80
<code>\StrBehind</code>	679, 712
<code>\StrDel</code>	940
<code>\StrSubstitute</code>	612, 613, 781, 782
T	
<code>\thefield</code>	753, 757, 758
<code>\tl</code>	17
U	
<code>\unexpanded</code>	136
<code>\unhbox</code>	736
<code>\unless</code>	925
<code>\unskip</code>	736
<code>\usebox</code>	918
<code>\usepackage</code>	881
<code>\uses</code>	6, 468
<code>\usescite</code>	6, 28, 489
<code>\usetikzlibrary</code>	869
V	
<code>\voidb@x</code>	737
W	
<code>\wd</code>	913, 915
Z	
<code>\z@</code>	186, 341, 918
<code>\zcref</code>	340

Change History

v0.1.0	General: Initial version	1
v1.0.0	General: First released version ...	1
v1.0.1	General: Capture <code>citet</code> and <code>citep</code> (including starred variants) like <code>cite</code> when they are defined ...	1
	Fix an infinite loop when, with the <code>cite</code> option, a citation occurred in a proof nested within another proof: the capture no longer reinstalls over itself	1
	Support, with the <code>cite</code> option, the two-optional-argument citation form [pre][post]{key} of natbib and biblatex; it previously broke compilation with confusing errors inside proofs ..	1
v1.1.0	General: Add <code>proofgraphignorecite</code> , dropping the dependency of a result on a cited work, which <code>proofgraphignore</code> cannot name, both of its arguments being labels	1
	Add <code>proofgraphprovenv</code> , declaring further environments to be treated as proofs, for a document whose proofs do not live in an environment named <code>proof</code>	1
	Add <code>usescite</code> and <code>proofgraphciteedge</code> , recording a dependency on cited work by hand, for a proof that does not cite it visibly and for a result stated without a proof to hold a citation	1

Capture <code>zcref</code> of <code>zref-clever</code> , in its starred, optional-argument and multiple-label forms, like the other cross-referencing commands	1
Capture citations, and not only cross-references, in the optional title of a result	1
Capture cross-references and citations in the body of a result and not only in its proof and its optional title, a statement resting on an earlier result being a dependency like any other; the new <code>statements</code> option, on by default, turns this off	1
Capture, with the <code>cite</code> option, every citation command of <code>natbib</code> and <code>biblatex</code> , such as <code>textcite</code> and <code>parencite</code> , down to those printing a mere fragment of a citation such as <code>citeauthor</code> , and add <code>proofgraphcitecommand</code> to capture further ones	1
Draw a result stated inside a proof, a claim settled in the course of an argument, as supporting the result that proof establishes. It was drawn with its own proof below it but nothing above, detached from the result it was raised to establish	1
Emit citation nodes again when <code>citelabel=tag</code> finds no tag to use. A stray <code>iftrue</code> , counted as a conditional while a false conditional was being skipped over, made the skip overshoot and swallow the writes emitting the node, which Graphviz then drew from its edges alone: unstyled, labelled by its internal name and without a hyperlink. It affected every first run, and <code>biblatex</code> documents always	1
Keep the braces of an optional title that is a single brace group, the bracing the <code>amsthm</code> manual prescribes to hide the] of a citation note. A title such as <code>[[cite[p. 37]{a-1}]]</code> reached the heading unbraced, whose own scan for] then stopped inside the note: the citation vanished from the printed title and two errors were reported against the capture	1
Let <code>proofof</code> pin a proof as a whole, wherever in the proof it stands: the references made before it went to whichever result preceded the proof, which a proof deferred to an appendix made easy to hit. The result it names may now be labelled further on in the document	1
Let a <code>proofof</code> in a proof reach the proofs nested in it, the steps of a structured proof: a nested proof took as its result the one that preceded the proof around it, so the references it made escaped the pin. A nested proof stated after a claim still proves that claim . . .	1
Point the hyperlink of a citation node at the destination <code>biblatex</code> gives a bibliography entry, which carries the number of the reference section before the key, rather than at the one the LaTeX kernel and <code>natbib</code> give, which <code>biblatex</code> never defines: the node led to the end of the document instead of the entry . .	1
Record no dependency while the page is assembled: a cross-reference or a citation in a sectional title is executed again in the running head, and a page breaking inside a proof made it a dependency of the result that proof belonged to . .	1
Recover the citation tag of the numeric and alphabetic <code>biblatex</code> styles for <code>citelabel=tag</code> , which records no citation tag in the <code>.aux</code> for it to read	1
Report an editing rule that dropped nothing, <code>proofgraphignore</code> above all, whose two labels go the way	

the dependency is recorded, as in proofgraphedge , and not the way the arrow is drawn: a rule written the wrong way round changed nothing and said nothing. The warning names the rule to write instead. Say so in the manual as well, which described the arguments in terms of the edge drawn . . .	1
Require a LaTeX2e format of October 2020 or later, which the use of expl3 and NewDocumentCommand already implied	1
Set the note of a citation node inside its tag and after it, “[Knu68, pp. 23–27]”, the way a citation with a note is printed, and render its -- and	
---	as the dashes the document shows
Trim the spaces after the commas of the lists given to proofgraphtrack and proofgraphuntrack ; every name but the first was silently ignored	1
With autorun , run Graphviz only when the graph, the engine or the format has changed. Rendering at every compilation kept producing a different image, since Graphviz stamps a creation date into its cairo -based outputs, and latexmk would then rerun until it reported that too many passes were needed	1